

A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference

Received: 27 May 2023

Accepted: 10 July 2023

Published online: 10 August 2023



Manuel Le Gallo^{1,6}✉, Riduan Khaddam-Aljameh^{1,6}, Milos Stanisavljevic^{1,6}, Athanasios Vasilopoulos¹, Benedikt Kersting¹, Martino Dazzi¹, Geethan Karunaratne¹, Matthias Brändli¹, Abhairaj Singh¹, Silvia M. Müller², Julian Büchel¹, Xavier Timoneda¹, Vinay Joshi¹, Malte J. Rasch³, Urs Egger¹, Angelo Garofalo¹, Anastasios Petropoulos⁴, Theodore Antonakopoulos⁴, Kevin Brew⁵, Samuel Choi⁵, Injo Ok⁵, Timothy Philip⁵, Victor Chan⁵, Claire Silvestre⁵, Ishtiaq Ahsan⁵, Nicole Saulnier⁵, Vijay Narayanan³, Pier Andrea Francesc¹, Evangelos Eleftheriou¹ & Abu Sebastian¹✉

Analogue in-memory computing (AIMC) with resistive memory devices could reduce the latency and energy consumption of deep neural network inference tasks by directly performing computations within memory. However, to achieve end-to-end improvements in latency and energy consumption, AIMC must be combined with on-chip digital operations and on-chip communication. Here we report a multicore AIMC chip designed and fabricated in 14 nm complementary metal–oxide–semiconductor technology with backend-integrated phase-change memory. The fully integrated chip features 64 AIMC cores interconnected via an on-chip communication network. It also implements the digital activation functions and additional processing involved in individual convolutional layers and long short-term memory units. With this approach, we demonstrate near-software-equivalent inference accuracy with ResNet and long short-term memory networks, while implementing all the computations associated with the weight layers and the activation functions on the chip. For 8-bit input/output matrix–vector multiplications, in the four-phase (high-precision) or one-phase (low-precision) operational read mode, the chip can achieve a maximum throughput of 16.1 or 63.1 tera-operations per second at an energy efficiency of 2.48 or 9.76 tera-operations per second per watt, respectively.

The need to repeatedly shuttle around synaptic weight values from memory to processing units is a key source of energy inefficiency in the hardware implementation of artificial neural networks¹. Analogue in-memory computing (AIMC) with spatially instantiated synaptic

weights could overcome this challenge, by directly performing matrix–vector multiplications (MVMs) within the network weights stored on a chip to execute an inference workload^{2–7}. Early work on performing neural network inference with AIMC showed promising accuracy results in

¹IBM Research Europe, Rüschlikon, Switzerland. ²IBM Systems and Technology, Böblingen, Germany. ³IBM Research, Yorktown Heights, NY, USA.

⁴University of Patras, Rio Achaia, Greece. ⁵IBM Research, Albany, NY, USA. ⁶These authors contributed equally: Manuel Le Gallo, Riduan Khaddam-Aljameh, Milos Stanisavljevic. ✉e-mail: anu@zurich.ibm.com; ase@zurich.ibm.com

mixed hardware/software implementations, where functionalities such as digital-to-analogue and analogue-to-digital conversions, activation functions and other necessary digital operations were implemented with off-chip software or hardware^{8–12}.

Various single- or few-core chips using static random-access memory (SRAM)¹³, flash¹⁴, resistive random-access memory (RRAM)^{15–17}, phase-change memory (PCM)¹⁸ and magnetoresistive random-access memory¹⁹ technologies have since been fabricated with fully integrated data conversions and sometimes activation functions^{14,15,18}. These have demonstrated a competitive energy efficiency of more than 10 tera-operations per second per watt (TOPS W^{−1}) for reduced-precision MVM. In such work, only small networks (less than 100k weights) on simple benchmark tasks such as MNIST digit recognition have been implemented to entirely fit on the core^{14–18}; larger networks have been emulated by programming each layer one at a time onto the same core^{13,19}. Recently, multicore chips supporting larger networks (more than 1M weights) have demonstrated promising inference accuracy results on more difficult benchmarks (such as image recognition on the CIFAR dataset) and high MVM energy efficiencies^{20–25}.

Nevertheless, to build a complete AIMC accelerator capable of end-to-end inference tasks with accuracy and efficiency that outperform existing digital accelerators, considerable challenges remain to be addressed. Because of the rather large size of the multibit SRAM plus switched-capacitor unit cells, existing SRAM-based implementations need off-chip weight buffers to hold network weights, which are then partially transferred to the AIMC cores because an entire network cannot fully fit on the chip²⁴. The current implementations are not dense enough to hold more than 10 MB of weight data in a reasonable chip area, limiting their efficiency gains over digital approaches to rather small networks that fit entirely on the chip¹. Another limitation is the volatility of the SRAM cells, which prevents the on-chip weights to be retained throughout the power cycling of the accelerator.

The potential to achieve smaller unit-cell sizes with resistive memory technologies could make it possible to hold such large networks entirely on the chip in a non-volatile way and eliminate the overhead of off-chip weight data access. In this context, an AIMC accelerator based on such devices is the natural choice. Both digital^{20,22} (one device corresponds to one weight bit) and analogue (one device encodes an entire weight)^{21,25} storage of weights in those devices have been explored in recent AIMC chips. Digital storage offers high accuracy, but the throughput is limited due to the small number of rows that can be operated at a time (sometimes as low as 4)²² without degrading the signal quality, to prevent large errors when processing the most significant bits. This approach also limits the achievable weight density because several devices are needed to encode a weight. Analogue weight storage offers high weight density and the ability to simultaneously operate more rows (demonstrated up to 512)²⁵. However, this approach suffers from accuracy degradation because of the noisy analogue weights and higher latency due to the need of slow high-resolution analogue-to-digital converters (ADCs) that often have to be multiplexed across columns. Moreover, extensive network-specific chip tuning, such as the per-network recalibration of the chip to mitigate non-idealities of the devices and peripheral circuitry, and chip-in-the-loop fine-tuning of weights through back-propagation, has so far been necessary to achieve acceptable inference accuracy with this approach^{11,21}. Furthermore, at the system level, none of the current AIMC chips based on resistive memory support all the computations involved in processing convolutional and long short-term memory (LSTM) layers.

In this Article, we report the IBM HERMES project chip: a 64-core AIMC chip based on backend-integrated PCM in a 14 nm complementary metal–oxide–semiconductor process. The chip delivers four key advances over previous implementations. First, high weight capacity is enabled through the analogue storage capability of PCM,

using only four PCM devices to encode a weight. Second, low MVM latency, high parallelism and high throughput are achieved by using compact current-controlled oscillator ADCs placed on each unit-cell row to perform fully parallel MVMs. Third, high MVM and inference accuracy are achieved without fine-tuning the pretrained weights when programming them onto the chip, owing to proper on-chip ADC calibration circuits that remove unwanted nonlinear response. Fourth, all the non-MVM computations involved in processing convolutional and LSTM layers are supported using dedicated high-precision digital units.

With the approach, we report measured inference results at near-software-equivalent accuracy on CIFAR-10 image recognition, Penn Treebank (PTB) character prediction and image caption generation tasks, with all the convolutional and LSTM layer computations performed on the chip. In its current form, the chip requires off-chip processing for interlayer connectivity to realize more than a single layer of a network, but it performs all intralayer computations on the chip, whether within a core or across neighbouring cores. The measured MVM throughput per area of the chip is more than 15 times higher than comparable multicore resistive memory AIMC chips, while achieving the highest accuracy on the CIFAR-10 benchmark.

Chip architecture

Figure 1a shows a snapshot of the chip inside the electronic design automation tool. The chip has a square dimension of 12 mm × 12 mm. Its 64 cores, each of size 1.20 mm × 1.16 mm, are placed in eight rows and eight columns. This subdivided structure is detailed in a schematic (Fig. 1b). Each core contains a PCM crossbar array capable of storing a 256 × 256 weight matrix and performing an analogue MVM using the input activations provided from outside the core. Hence, up to 4,194,304 weights can be stored on the chip for performing in-memory MVMs.

At the centre of the core grid, between rows 4 and 5, a row of eight global digital processing units (GDPUs) is placed. They provide the additional digital post-processing capabilities needed when running LSTM networks. The high-frequency clock signals for the digital circuitry in the cores and GDPUs are received at the centre of the chip and are distributed in a balanced tree across the chip.

Finally, the core outputs and GGPU inputs are interconnected by a grid of horizontal and vertical digital communication links (Fig. 1b), yielding a deep neural network acceleration architecture where the weight elements are stationary, and only the activation vectors are transmitted. There are 418 physical communication links in total, which together implement a simplified version with fewer bidirectional links of the 5 Parallel Prism communication fabric topology²⁶ (Extended Data Fig. 1 shows a map of all the possible link connections). Extended Data Table 1 lists a summary of the chip specifications.

The computational memory core

Figure 1c depicts the architecture of a single core and its various components. The design is based on the fully integrated standalone PCM-based single-core chip presented elsewhere¹⁸, with the addition of the link controller for transmitting and receiving data across different cores. At the centre of the core, there is a 256 × 256-sized crossbar array of PCM-based unit cells. As shown in Extended Data Fig. 2a, four PCM devices are used per unit cell: two each for each polarity, such that positive and negative weights can be represented^{27,28}. The phase configuration of the phase-change material within the mushroom-type PCM devices determines their conductance (Extended Data Fig. 3). The devices in a conductance pair are weighted with equal significance.

To program the individual conductances, a diagonal decoding scheme²⁸ is employed to select 256 unit cells, one per row and column. The whole procedure starts with the dedicated per-core programming finite-state machine, instructing the diagonal selection decoder to enable one diagonal of cells that contains the devices that are to be

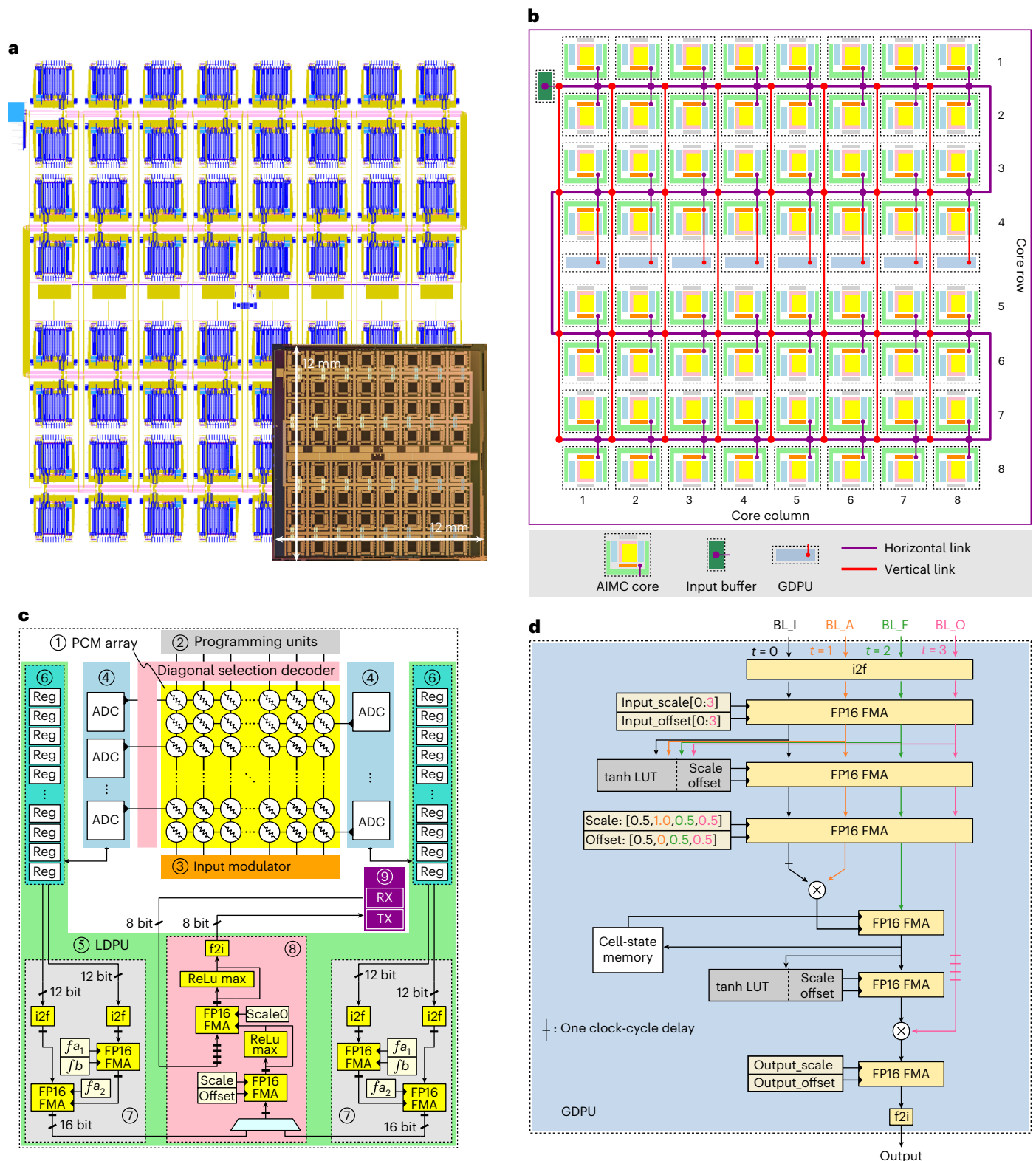


Fig. 1 | IBM HERMES project chip overview. **a**, Electronic design automation snapshot; the inset shows a micrograph of the chip. Here the outline of the 64 cores can be recognized as well as the array of 5,616 pads. **b**, Schematic of the overview of the different components on the multicore chip. **c**, Schematic of the overview of a single PCM-based in-memory compute core. (1) PCM crossbar array, (2) current DAC-based programming unit, (3) PWM-based input modulator, (4) left and right ADC arrays, (5) LDPUs, (6) left and right ADC register arrays, (7) left and right ADC convert-and-scale blocks, (8) activation function block and (9) link controller. **d**, Block diagram of a GDPU used for LSTM-related data processing. Inputs to and outputs from the GDPU slice are in an 8-bit signed

integer format (INT8). By using custom conversion blocks marked by i2f (integer to float) and f2i (float to integer), INT8 values can be converted into the half-precision floating-point format (FP16) and vice versa. Additionally, conversions at the input/output can encompass a per-gate/per-output scale and bias operation using the FMA units. The inputs from the I (input gate), A (cell input), F (forget gate) and O (output gate) BLs are time multiplexed, and a single block is used to compute the gates' activation vectors. The sigmoid activation function for the I, F and O gates is computed by scaling and offsetting the output of the hyperbolic tangent function with the third (from the top) FMA unit, according to the identity $\text{sigmoid}(x) = 1/2 + 1/2 \times \tanh(x/2)$.

programmed (Extended Data Fig. 2b). Subsequently, these selected devices can be programmed by the current-steering digital-to-analogue converter (DAC)-based programming units located on top of the PCM array. The DAC units are capable of producing all the required programming pulse shapes for the PCM devices based on the instruction provided by the finite-state machine. This includes high-current RESET pulses, triangular-shaped SET pulses and programming pulses with variable current amplitudes and time durations.

Once all the programming operations are concluded, the core can be used to perform analogue MVMs. To this end, the input modulator applies pulse-width-modulated (PWM) read-voltage pulses to the PCM array, the output of which is digitized by an array of 256 time-based current ADCs. These ADCs flank the PCM array from the left and right sides, such that each unit-cell row is associated with a dedicated ADC. This is achieved by matching the ADC height to that of two unit cells, which results in a skewed aspect ratio (Extended Data Fig. 2c shows the layout). By choosing a current-based ADC topology, area-intensive and noise-prone current-to-voltage conversion steps can be avoided. Furthermore, by opting for a time-based architecture, a highly digital design is obtained in which the output precision can be adjusted by changing the conversion time.

To counteract the manufacturing mismatch and other non-idealities, the ADC includes an internal resistor ladder-based voltage DAC that allows correcting the read-voltage offset¹⁸. Furthermore, the internal current mirror, which drives the attached current-controlled oscillator unit, contains trimming registers that allow matching the gain between the different ADCs per core and compensating nonlinearity in their transfer function¹⁸. Thus, after performing an initial calibration procedure, the configuration parameters are stored on the chip such that the ADCs exhibit tightly distributed and linear transfer curves. Note that this calibration is independent of the deployed application and does not have to be redone for different weight sets or networks.

Extended Data Fig. 4a illustrates one row of the computational memory during an analogue MVM. The per-row ADC regulates the bit line (BL) to a fixed common-mode voltage, such that the top electrodes of all the PCM devices in the 256 unit cells are on the same potential. Their respective source-line (SL) potentials, however, are connected to one of the three potentials based on the respective input value, that is, the positive read voltage V^+ to subtract current, the negative read voltage V^- when adding current or high impedance when no current from the cell is to be added. Thus, all the combinations of products between a positive or negative weight and a positive or negative input can be performed in the analogue domain in a single modulation (Extended Data Fig. 4b). In addition, a four-phase mode is supported (Extended Data Fig. 4c). Here the four combinations of $\pm$ weights and $\pm$ inputs are performed in four separate operations and only using the negative read voltage V^- . As a result, a higher accuracy of the measured current is achieved, as all the PCM devices are read in the same polarity by the current-based ADC. Therefore, all the experiments in this work were performed using the higher-precision four-phase mode. Compensation schemes to mitigate the impact of the current–voltage polarity dependence of PCM²⁹ to enable high precision in the one-phase mode will be the subject of future investigations.

The ADC delivers the final results as two 12-bit unsigned integers, one for the positive and one for the negative current. By using tristate buffers on a 24-bit-wide bus, the results are transmitted from each ADC to the local digital processing unit (LDPU) for post-processing. In the first step, the results are transferred to a register array, such that the operation becomes fully pipelined. Consequently, the initial latency of the LDPU does not affect its throughput (one data point per clock cycle). Next, the results are transferred one by one to the ADC convert-and-scale block, which first converts the 12-bit integers into half-precision floating-point format (FP16) and then employs two fused multiply–add (FMA) units to remove gain and offset errors.

Two ADC convert-and-scale blocks produce outputs every second clock cycle and feed them one after the other to the activation function block through a multiplexer unit. Here affine scaling per channel and an optional rectified linear unit (ReLU) operation are applied. The intermediate value is then combined with an 8-bit signed integer (INT8) input coming from the neighbouring cores, or from off the chip, through the link controller receiver. This value is first converted into the FP16 format and then scaled to the optimal range. Finally, the output undergoes another optional ReLU operation before conversion into INT8, ready to be shipped by the transmitter of the link controller. The existence of both ReLU operations allows the possibility to apply the activation either before or after adding the receiver link input. The activation block is fully pipelined so that its latency does not affect the LDPU throughput.

The link controller implements a state machine that enables the transmission and reception of data within at most six neighbouring cores (receiving and transmitting cores are not necessarily the same; Extended Data Fig. 1). The core-to-core input–output links are implemented as eight parallel channels, each transferring one bit every clock cycle. The selection of the data source for the transmitter and of the data destination for the receiver is implemented through a routing table, configured before runtime (Extended Data Fig. 1a). In the current design, the links can transfer data across the LDPUs of multiple cores to realize fully on-chip intralayer partial sum accumulation, and transfer data from the LDPU to the input of a GDPU. Data transfer from the LDPU of one core to the input modulator of another core—required to send the activations of one layer to another layer—is currently not supported and is therefore implemented through off-chip communication.

The GDPU

The GDPU is responsible for the digital processing required in LSTM networks. The chip contains eight GDPU slices in total, each connected to the fourth row core immediately above. The cores in a column above a GDPU slice can aggregate their outputs in a cascaded fashion in the LDPUs of those cores. The final aggregated output produced by the LDPU of the fourth row core is then sent to its corresponding GDPU slice. Each slice serially takes up to 256 inputs (one in each clock cycle), composed of 64 preactivation inputs to the input gate (I), cell input (A), forget gate (F) and output gate (O). Those preactivation inputs arrive in an alternating fashion from four neighbouring BLs. By mapping the columns of the I, A, F and O weight matrices in an interleaved fashion, a single element processor per GDPU slice (instead of 64) can be used to serially process the 256 inputs, which greatly reduces the number of resources needed in the GDPU. Each GDPU slice produces one output every four clock cycles for up to 64 outputs. It also contains the cell-state memory registers for up to 64 elements.

Figure 1d depicts one GDPU slice architecture in detail. All the computations inside the GDPU slice are performed in the FP16 format, with the inputs and outputs in the INT8 format. Hyperbolic tangent activation functions are realized using a lookup table (LUT) method. First, a bank of 17 comparators is used to find the corresponding bin (out of 18) in which an input is contained. Then, the output value is interpolated in the FMA unit using slope-and-offset read from the LUT for the determined bin. The output activations of the four gates are further processed by two multiply blocks, three FMA blocks and one more hyperbolic tangent LUT block to compute the cell state and the hidden-state output of the LSTM unit.

MVM accuracy

We characterized all the 64 cores of a single chip to assess the PCM yield and how accurately MVMs can be performed with them. Figure 2a shows the SET and RESET distributions of the unit cells of all the 64 cores. The unit cell comprises four PCM devices (two per polarity).

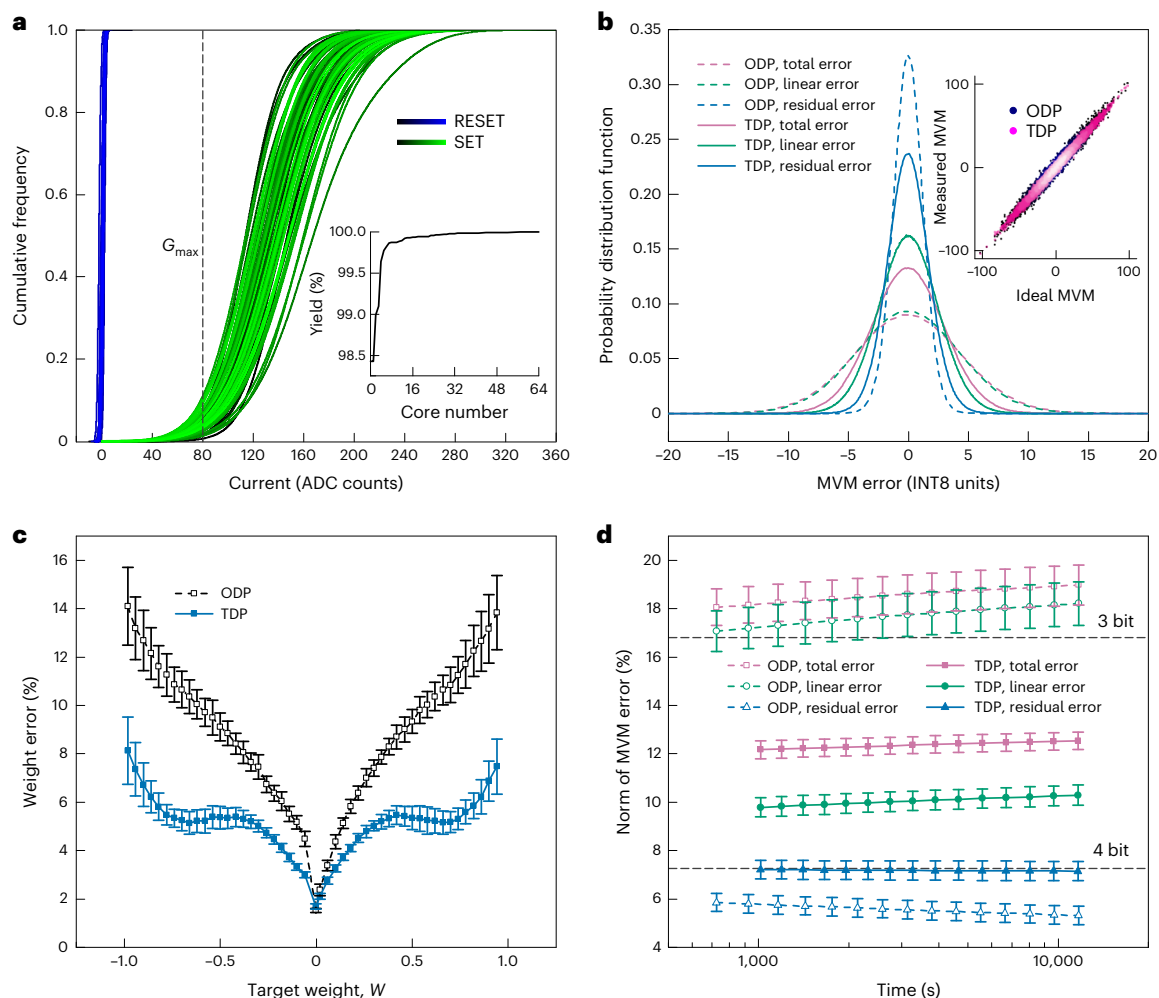


Fig. 2 | MVM characterization. **a**, Unit-cell SET and RESET distributions of the 64 cores. The shades of blue and green denote different cores. The dashed line represents the ODP $G_{\max}$, defined as the tenth percentile of the core with the least conductive SET states. The inset shows the unit-cell yield of the 64 cores. The yield condition is that the unit cell can be programmed to $|G_{\text{RESET}}| < 5$ and $G_{\text{SET}} > 50$. **b**, Error distributions of ϵ_{total} , ϵ_{linear} and $\epsilon_{\text{residual}}$ for the 64 cores in the LDPU (INT8) units. A uniformly distributed weight matrix with 30% sparsity is programmed on each core, and 2,048 input vectors uniformly distributed with 30% sparsity are then sent to each core to perform the MVMs. The reduction in ϵ_{total} achieved with TDP can be attributed to a reduction in ϵ_{linear} . The distributions are computed over all the error vector elements of 2,048 MVMs performed with all the 64 cores.

The inset shows the measured MVM results of one core for ODP and TDP against the ideal MVMs computed in software. **c**, Weight error $\text{std}(W - \hat{W})/W_{\max}$, where $\text{std}(W)$ is the standard deviation computed over all the elements of W , as a function of the target weight for ODP and TDP. Data are presented as mean values $\pm$ one standard deviation over the values from the 64 cores. **d**, The 2-norm of ϵ_{total} , ϵ_{linear} and $\epsilon_{\text{residual}}$, normalized by $\|V_{\text{IP}}\|_2$, as a function of time for ODP and TDP. Due to temporal conductance drift of the PCM devices, ϵ_{total} and ϵ_{linear} increase gradually. The dashed lines represent the error achieved by a digital engine with 8-bit input/output precision and N -bit weight precision. Data are presented as mean values $\pm$ one standard deviation over values from the 64 cores.

Therefore, we define a unit cell in the RESET state (G_{RESET}) when all the PCM devices are programmed to RESET, and a unit cell in the SET state (G_{SET}) when one PCM device is programmed to SET and the others to RESET. On 63 out of the 64 cores of this chip, more than 99% of the unit cells can be programmed to $|G_{\text{RESET}}|$ smaller than 5 ADC counts and G_{SET} larger than 50 ADC counts. The yield of the remaining outlier core is 98.4%. The high yield achieved on all the cores implies that it is possible to program most of the unit cells to a given conductance value via iterative programming, as long as it is within the range of their G_{RESET} and G_{SET} values.

We studied two programming algorithms using either strictly one-device programming (ODP) or up to two-device programming (TDP) to map a weight to a unit-cell conductance. A weight is written by programming the unit-cell devices that correspond to the weight polarity. Devices of the opposite polarity are RESET to near-zero conductance. For a given unit cell at row m and column n in a core, the corresponding weight W_{mn} is mapped to a target conductance

value G_{mn} , corresponding to the double differential construct of the four-PCM-device conductance values according to

$$G_{mn} = W_{mn} \times \frac{G_{\max}}{W_{\max}}, \quad (1)$$

where $W_{\max}$ is the maximum absolute weight value to be programmed in the core and $G_{\max}$ is the maximum reliably programmable unit-cell conductance. For ODP, $G_{\max}$ should ideally not be larger than the least conductive SET device of the core. We define $G_{\max} = 80$ ADC counts based on the SET distributions of Fig. 2a, as the tenth percentile of the core with the least conductive SET states. For TDP, $G_{\max}$ should be set according to the least conductive unit cell when both devices with positive or negative polarity are in the SET state. Here we choose $G_{\max} = 160$ ADC counts.

We use a closed-loop iterative programming scheme in which the programming current is updated proportional to the difference between the read conductance of unit cell (m, n) and G_{mn} (ref. 30). In each

iteration, the devices are individually read, and a programming pulse is applied to the unit cells that did not yet converge within a predefined margin. For ODP, one out of the two unit-cell devices corresponding to the weight polarity receives the iterative programming pulses, and the other is RESET. For TDP, if G_{mn} exceeds G_{SET} of both devices, one is programmed to the SET state and the other receives the iterative programming pulses. Otherwise, the device with the highest G_{SET} value receives the iterative programming pulses and the other is RESET (Methods and Extended Data Fig. 5). Performing the device selection this way ensures that the maximum amount of devices are at RESET or SET, which are the least noisy states. Because of this, reduced programming errors are achieved compared with prior multidevice programming approaches that attempt to program each unit-cell device via iterative programming up to the ODP G_{max} (ref. 31) (Extended Data Fig. 5c).

To quantify the MVM accuracy and precision achieved with ODP and TDP on the 64 cores, a randomly distributed weight matrix W is programmed on each core, and 2,048 randomly distributed input vectors x are then sent to each core to perform the MVMs. Based on the measured MVM results y_{exp} , the actually programmed weights can be estimated as $\hat{W} = \operatorname{argmin}_{\hat{W}} \|y_{exp} - \hat{W}x\|_2$. We then characterize the MVM error using the three following metrics:

$$\epsilon_{total} = y_{exp} - y_{fp}, \quad (2)$$

$$\epsilon_{linear} = \hat{W}x - y_{fp}, \quad (3)$$

$$\epsilon_{residual} = y_{exp} - \hat{W}x, \quad (4)$$

where $y_{fp} = Wx$ denotes the MVM result computed in double-precision floating-point format. Also, ϵ_{total} is the total MVM error, which we decompose into a linear contribution ϵ_{linear} and a residual contribution $\epsilon_{residual}$. In the first approximation, ϵ_{linear} defines the error resulting from a wrong programming of the weights. Then, $\epsilon_{residual}$ results from any residual error of the chip that cannot be cast as a weight error, such as PWM and ADC nonlinearities, instantaneous read noise, leakage current and IR drop (parasitic voltage drop between two ends of a conducting wire of resistance R when current I flows through it).

The error histograms of the three metrics over all the 64 cores are plotted in Fig. 2b for ODP and TDP. For ODP, ϵ_{total} is largely defined by ϵ_{linear} , and $\epsilon_{residual}$ is comparably negligible. When using TDP, ϵ_{linear} is substantially reduced compared with ODP, showing that the weight programming error is effectively lowered when using more devices per weight. In this case, the impact of $\epsilon_{residual}$ gets noticeable in the total error. We ascribe the wider spread of $\epsilon_{residual}$ for TDP to the fact that a larger range of the ADC transfer curve and particularly the more nonlinear high-current range are sampled. Moreover, higher leakage from the PCM array than ODP is expected because more devices are programmed to the non-RESET states, which also increases $\epsilon_{residual}$. Figure 2c shows the weight error plotted as a function of the target weight, confirming that TDP allows this to be reduced for all the non-zero weight values. We attribute this reduction to a higher signal-to-noise ratio achieved when using two devices instead of one, because the programming errors of the individual devices combine with lower variance in the average of their conductance values³¹.

For resistive memory technologies, it is also critical to study the degradation of the MVM accuracy over time. In PCM, the conductance values drift with time t according to $G(t) = G(t_0)(t/t_0)^{-\nu}$, where $G(t_0)$ is the conductance at t_0 and ν denotes the drift exponent^{32,33}. Drift can be partially mitigated by a global drift compensation procedure, that is, an appropriate per-core rescaling of the MVM result^{31,34}. However, since each programmed weight drifts with a slightly different rate (ν depends on the conductance state and varies from device to device), ϵ_{linear} and consequently ϵ_{total} increase over time (Fig. 2d). The subtle reduction in $\epsilon_{residual}$ can be ascribed to the sampling of a gradually smaller range of

the ADC transfer curve. Compared with an equivalent digital engine having 8-bit input/output precision and N -bit weight precision, the precision achieved with ODP is close to 3-bit weights. With TDP, a precision between 3-bit and 4-bit weights is achieved.

Deep neural network inference demonstrations

Having established the MVM accuracy of the chip, we now assess the inference accuracy when running different neural networks on it. We implemented three networks that cover all the usable features of the chip: a ResNet-9 convolutional neural network (CNN) for the CIFAR-10 (ref. 35) image classification, an LSTM network for predicting characters of the PTB dataset³⁶ and an LSTM network for generating captions of images in the Flickr 8k dataset³⁷ (Methods). Before being deployed on the chip, the networks are trained in a hardware-aware manner by injecting noise on the synaptic weights to improve their resilience to hardware non-idealities^{12,38}, using the publicly available IBM Analog Hardware Acceleration Kit³⁹ (Methods provides details about the training procedure). The trained weights are then mapped to conductance values to be programmed on the PCM unit cells using equation (1). Here G_{max} is determined on a per-core basis and the BL current is constrained to not exceed the maximum supported by the ADC (Methods). For the three networks, all the operations involved in the processing of weight layers, including the MVMs, activation functions, batch normalization, LSTM cell-state computation, data aggregation when a layer is split onto multiple cores or for residual connections, bias addition and all the necessary intra- and interlayer affine scaling, are implemented on the chip. Apart from the max-pooling in ResNet-9 and the character/image/word embedding in the LSTM networks, which are not supported by the chip, no other computation is performed in software in the network executions. Only the communication of activation vectors between the output of a layer and the input of the next layer is done by off-chip data transport through the field-programmable gate array on the test board due to limitations in the communication fabric, and will be implemented on the chip in future designs.

The custom ResNet-9 network (1,866,536 synaptic weights and 10 biases) and its corresponding implementation on the chip are shown in Fig. 3a,b, respectively. On-chip links are configured to aggregate the MVM results in the LDPU of the cores implementing the layers that cannot fit onto a single one (all the layers except the first and the last). The batch normalization and ReLU activation for all the convolution layers are performed in the LDPU of the core that receives the aggregated data. The residual connections are implemented by sending the residual data to the LDPU of the receiving core, and performing the addition inside the LDPU. For the last dense layer, the bias is programmed into the affine-scale registers of the LDPU so that it gets added to the MVM result. The CIFAR-10 test accuracy results for ODP and TDP weight programming are shown in Fig. 3c. We achieve a hardware accuracy of 92.81% with TDP, which is less than 1.00% below the software baseline of 93.67%, whereas ODP achieves 92.23%. We also compare the experimental results with simulations that include the per-core weight error for this network (Fig. 2c) and the 8-bit quantization from the PWM and LDPUs (Methods shows the details on the simulation model). The biggest accuracy drop from the software baseline comes from the weight error, and the digital quantization adds another 0.2–0.3% drop on top of it (Fig. 3c). The accuracy obtained by simulating only those two effects nearly matches the experimental accuracy, which indicates that any additional nonlinearity from the peripheral circuitry of the chip does not have a notable impact compared with the weight and quantization errors.

The LSTM network for PTB character prediction (1,299,312 weights and 2,066 biases) is shown in Fig. 4a, and the corresponding implementation on the chip is shown in Fig. 4b. Each character is first embedded into a 128-long random orthogonalized vector, which is then input to the on-chip input gate. Then, MVMs from the input and hidden gates are vertically summed on the chip by the LDPUs of the eight cores in rows 3 and 4, using vertical on-chip links to transmit the data. The data

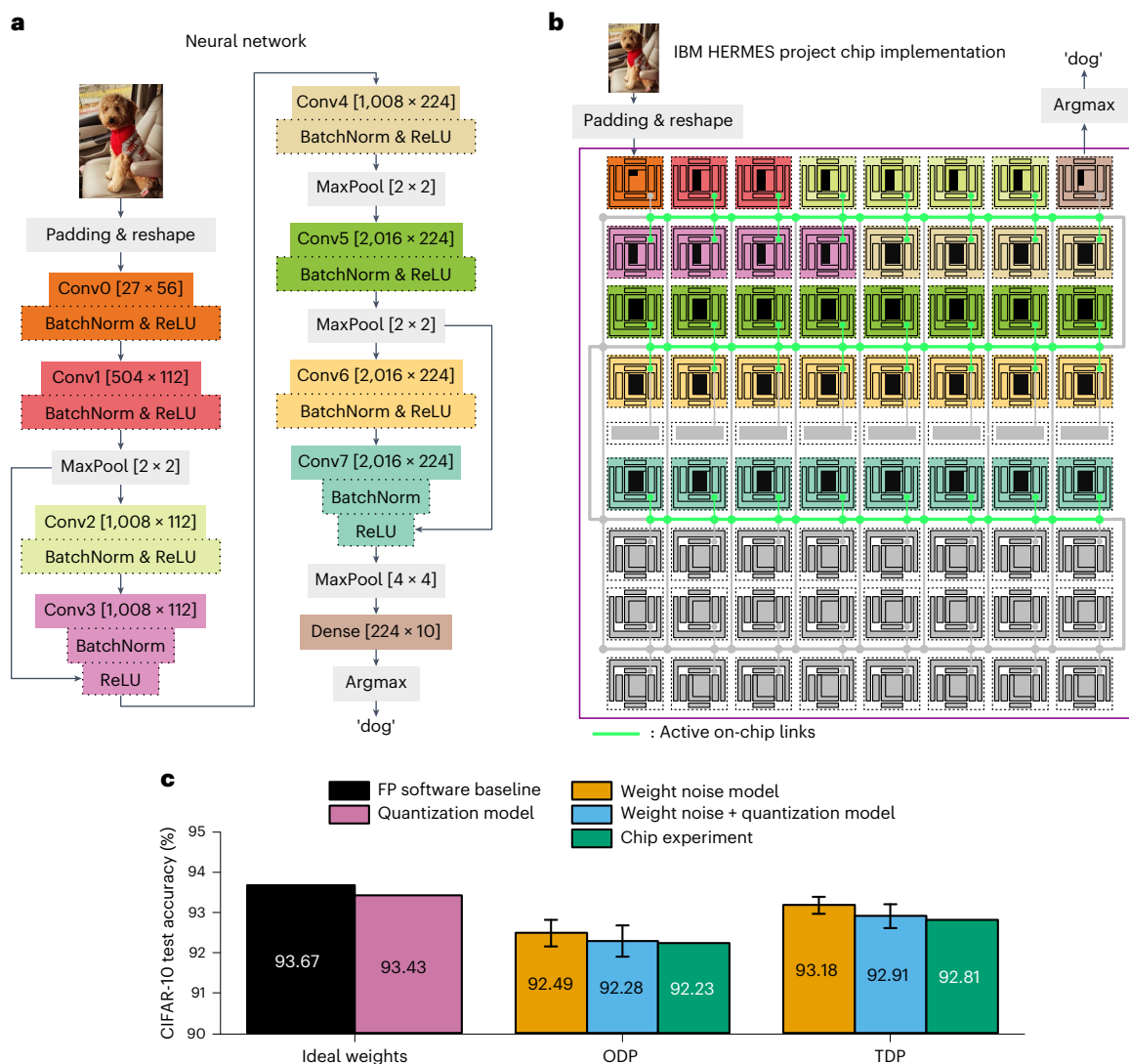


Fig. 3 | ResNet-9 on CIFAR-10 measurement results. a, Network architecture. ResNet-9 comprises eight convolution layers and one dense classification layer. Each convolution layer is followed by batch normalization and ReLU activation. Two residual connections from Conv1 to Conv3 and Conv5 to Conv7 are present. Four max-pooling layers, implemented off the chip, are used to reduce the size of the input volume along the network depth. **b**, Mapping of ResNet-9 onto the chip. Weights of all the layers are programmed onto 40 cores. Conv0–4 and the dense layer are implemented on the first two rows of the cores, whereas Conv5–7 each

span an entire row. On-chip links connect the LDPU of the cores within a layer to realize on-chip data aggregation. Batch normalization, ReLU, residual and bias additions are implemented in the LDPU. **c**, Measured test accuracy results on the CIFAR-10 benchmark for ODP and TDP compared with the software baseline and simulation results that include the hardware-measured weight noise and quantization from the PWM and LDPU. Simulation data are presented as mean values $\pm$ one standard deviation over ten inference runs.

produced by the LDPU of the eight cores of row 4 are then sent to the eight corresponding GDU slices using on-chip links. The GDU then performs the sigmoid and tanh activations, the cell-state computation as well as the necessary input/output data scaling. Finally, the output of the GDU is sent to the dense layer that outputs the next predicted character in the sequence, as well as to the hidden gate as input for the character of the next timestep. The performance of the character prediction task is evaluated in bits per character (BPC), which is a measure of how well the model is able to predict samples from the true underlying probability distribution of the dataset (a lower BPC corresponds to a better model). The BPC results for ODP and TDP weight programming are shown in Fig. 4c, as well as the corresponding simulation results. With TDP on hardware, we achieve a BPC less than 0.100 higher than the software baseline of 1.336 (adequate baseline on this benchmark for a network of this size⁴⁰), whereas the ODP BPC is only -0.020 higher than TDP. Unlike for ResNet-9, the high per-core utilization of the LSTM network leads to a high enough current at the ADC input with ODP to

achieve a good signal-to-noise ratio; therefore, using TDP only marginally improves the BPC. Here, again, the simulated accuracy with weight noise and quantization nearly matches the experimental accuracy.

The third network that we implemented is meant to assess the inference accuracy on a workload that uses all 64 cores. We chose an image caption generation task implemented with a single LSTM unit and a dense layer (4,080,384 weights and 6,080 biases), similar to the PTB LSTM network described above (Fig. 5a). Here, at the initial timestep, the features of an input image extracted by a CNN (implemented in software) are fed to the on-chip input gate. As above, the MVMs from the input and hidden gates are summed on the chip and sent to the GDU via on-chip links. The GDU output is sent to the dense layer that produces the first word of the caption, which is stored in an off-chip caption buffer. Then, this word is embedded, sent to the input gate and the GDU output is sent to the hidden gate, to generate the next word of the caption (Fig. 5b). The latter process is run until the entire caption is generated, for at most 24 timesteps per image

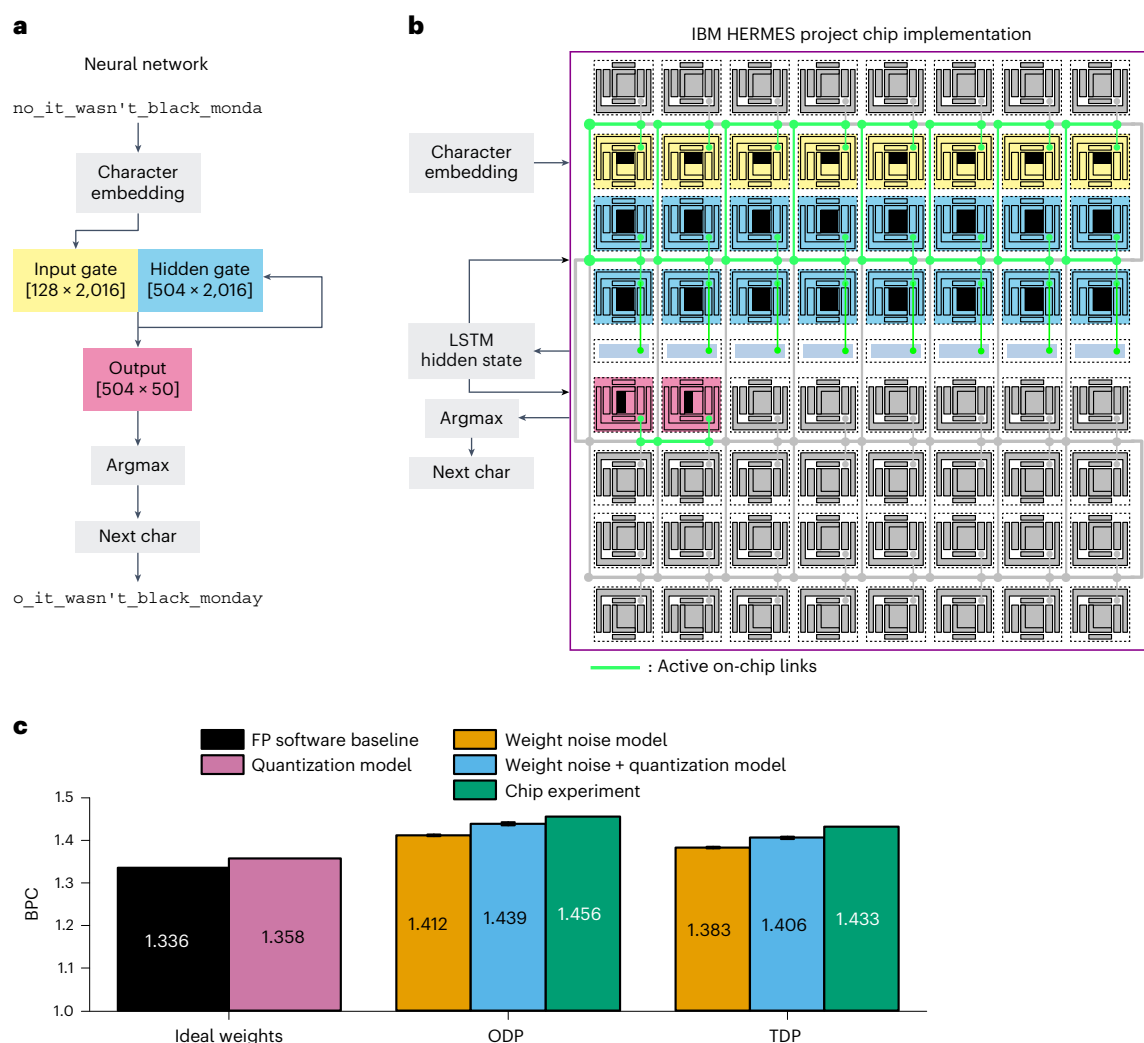


Fig. 4 | LSTM for character prediction measurement results. **a**, Network architecture. The LSTM unit comprises an input gate that receives the embedded input characters and a hidden gate whose input is the output hidden-state vector of the LSTM unit. The output layer is a dense classification layer whose output is the next predicted character. **b**, Mapping of the LSTM network onto the chip. Weights of all the layers are programmed onto 26 cores of the chip. Rows 2, 3 and 4 encode the weights of the LSTM unit and vertical on-chip links connect

their LDPU to realize on-chip data aggregation. The LDPU of the cores in row 4 are connected to their respective GDP slice via on-chip links. **c**, Measured BPC results for ODP and TDP compared with software baseline and simulation results that include the hardware-measured weight noise and quantization from the PWM, LDPU and GDP. Simulation data are presented as mean values $\pm$ one standard deviation over ten inference runs.

(Supplementary Video 1). The performance of the image caption generation task is evaluated using the BLEU- n score⁴¹, which measures the match of word n -grams between a generated caption and multiple reference captions. The BLEU scores measured over the full test dataset are shown in Fig. 5c, along with three representative generated captions in Fig. 5d. The BLEU scores achieved by the chip are essentially the same, within variation, as the software ones. It is observed that captions of some images differ between software and hardware, sometimes with one being better or worse than the other, as shown in the examples in Fig. 5d. However, when averaged over all the images of the dataset, the hardware BLEU scores are almost identical to the software ones.

Performance

Table 1 summarizes the measured performance achieved by the chip for four use cases: (1) the chip performing only MVMs at full utilization, (2) the processing of one input to a deep layer of ResNet-9, (3) the processing of one timestep of the LSTM unit of the PTB network and (4) the processing of one timestep of the LSTM unit of the image captioning network (Methods). Extended Data Fig. 6 shows the area and power

breakdown of the main chip components. A peak MVM throughput of 63.1 TOPS is achieved at an energy efficiency of 9.76 TOPS W^{-1} and MVM area efficiency of 1.55 TOPS mm^{-2} for the one-phase read mode, when all the 64 cores are fully utilized to perform MVMs in parallel. For the four-phase read mode, the former metrics are approximately four times lower due to the ~ 4 times higher latency. Even then, the raw MVM throughput of the chip in the four-phase read mode is higher than the previous state-of-the-art AIMC chips based on RRAM, PCM and SRAM, and is comparable with the NOR-flash chip²³, achieving the highest accuracy on the CIFAR-10 benchmark (Extended Data Table 2). Although the energy efficiency of the chip is generally lower, the higher throughput density outweighs the reduced energy efficiency by more than 1.8 times compared with all previous AIMC chips based on resistive memory (Extended Data Table 2). For example, the four-phase read throughput per area is 15 times higher than that in another work²⁰, but the energy efficiency is only 8 times lower. Although the SRAM-based accelerator presented in another work²⁴ has a higher energy efficiency as well as throughput density, it requires the use of an off-chip weight buffer to store the total weight data of a network, leading to additional

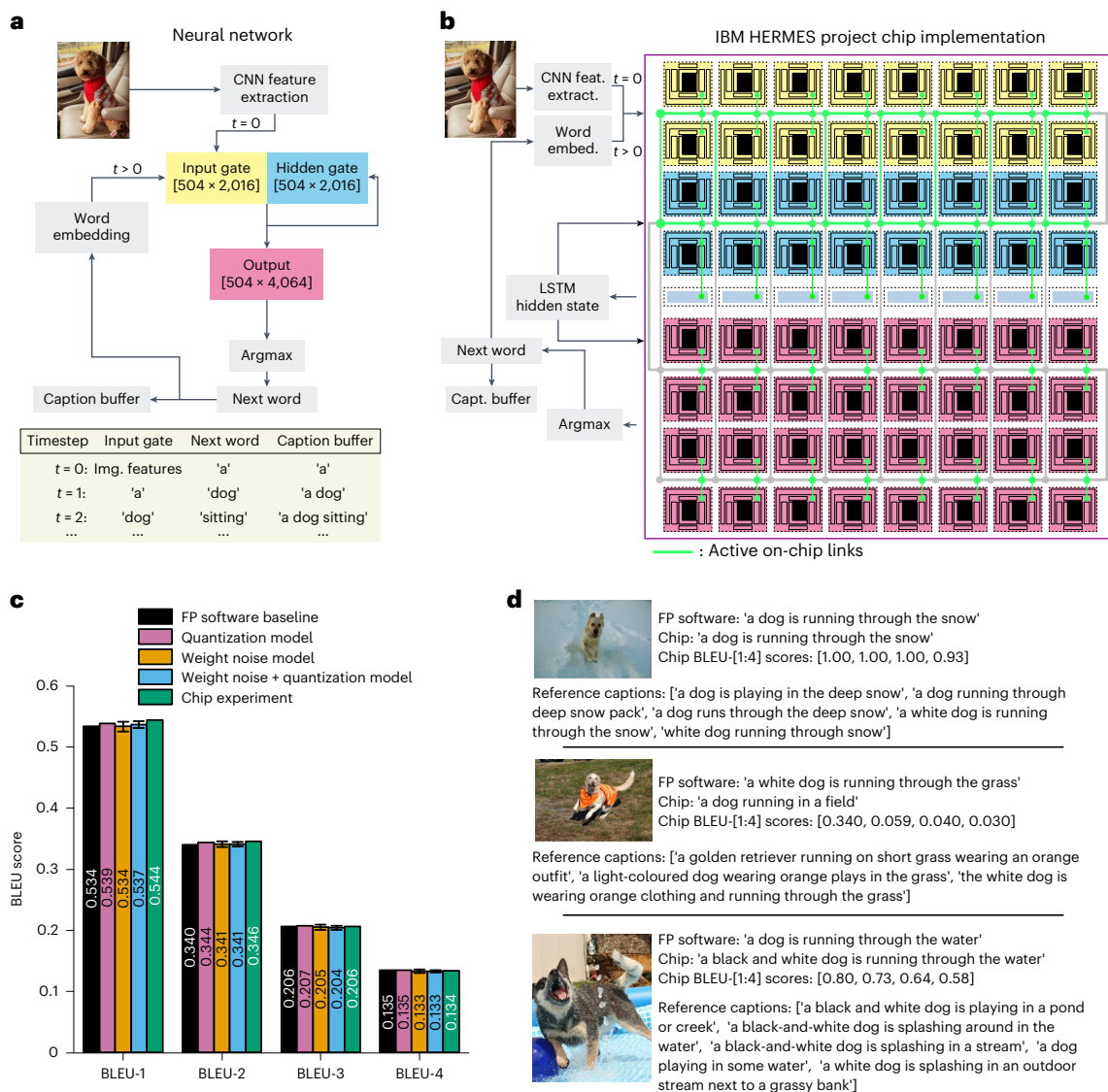


Fig. 5 | LSTM for image caption generation measurement results. a, Network architecture. The features of the input image extracted by a CNN (off chip) are fed to the input gate of the LSTM unit. The output hidden-state vector of the LSTM unit is sent to the output dense layer that generates the first word of the caption, which is stored in a buffer. This word is then embedded and fed back to the input gate of the LSTM unit to generate the next word. **b**, Mapping of the network onto the chip. Weights of LSTM and dense layers are programmed onto all the 64 cores. Rows 1–4 encode the weights of the LSTM unit and vertical on-chip links connect their LDPU to realize on-chip data aggregation. The LDPU of the cores in row 4 are connected to their respective GGPU slice via on-chip links. The

dense layer is mapped on rows 5–8, and on-chip vertical data aggregation is set up within rows 5 and 6 and rows 7 and 8. **c**, Measured BLEU-*n* scores from the chip with ODP compared with software baseline and simulation results that include the hardware-measured weight noise and quantization from the PWM, LDPU and GGPU. Only ODP was used for this network because $G_{\max}$ cannot be increased beyond 80 ADC counts to stay within the linear region of the ADC; therefore, TDP does not bring accuracy benefits (Methods). Simulation data are presented as mean values $\pm$ one standard deviation over ten inference runs. **d**, Comparison between software- and chip-generated captions for three example images, along with the reference captions over which the BLEU scores are computed.

overhead during inference execution. In contrast, our implementation enables a model to be stored entirely on the chip, thereby reducing external memory accesses during execution as well as being able to retain the weight data throughout power cycling.

Finally, regarding the efficiency of the neural network layers including the digital operations and on-chip data aggregation through links, a single input to the ResNet-9 layer is processed in 1.52 μ s and consumes 1.51 μ J, whereas a single timestep of the LSTM unit of the image captioning network is executed in 1.43 μ s and consumes 5.24 μ J. Reducing the precision of the digital computing from FP16 to a lower-integer precision, such as INT8 (ref. 42), will allow—in the future—the digital computations to be parallelized across ADC outputs to reduce the latency gap with the MVM-only latency.

Conclusions

We have reported the IBM HERMES project chip: a 64-core AIMC chip based on 14 nm complementary metal–oxide–semiconductor with backend-integrated PCM that can be used for deep neural network inference tasks. The chip can implement more than 4M weights split across 256×256 cores, perform MVMs through AIMC and support all non-MVM computations involved in convolutional and LSTM layers via dedicated digital units. The chip achieves a peak MVM throughput of 16.1–63.1 TOPS at an energy efficiency of 2.48–9.76 TOPS W^{-1} in the four-phase (high-precision)–one-phase (low-precision) read mode for 8-bit input/output MVMs. The measured MVM throughput per area of 400 giga-operations per second per square millimetre in the four-phase read mode is more than 15 times higher than previous multicore AIMC

Table 1 | Measured speed and energy efficiency of the IBM HERMES project chip

	MVM at maximum utilization		ResNet-9 layer		One timestep of LSTM unit (PTB)		One timestep of LSTM unit (image captioning)	
Number of cores used	64		8		24		32	
Average core utilization	100%		86%		81%		97%	
Read mode ^a	1 phase	4 phases	1 phase	4 phases	1 phase	4 phases	1 phase	4 phases
Peak MVM throughput (TOPS) ^b	63.1	16.1	6.79	1.74	19.2	4.90	30.6	7.82
Peak MVM energy efficiency (TOPS W ⁻¹) ^b	9.76	2.48	6.88	1.74	7.74	1.96	9.34	2.37
MVM area efficiency (TOPS mm ⁻²) ^b	1.55	0.40	1.34	0.34	1.26	0.32	1.50	0.38
Energy (μJ)	0.86	3.38	0.97	1.51	2.56	3.92	3.46	5.24
Latency (ns)	133	520	1,131	1,518	1,043	1,430	1,043	1,430

^a All MVM and inference accuracies are reported for the four-phase mode. ^b Throughput, energy and area consider only the MVM operations performed by the cores. One multiply-and-accumulate operation = two operations.

chips based on resistive memory²⁰, while achieving a CIFAR-10 accuracy of 92.81%. Our approach also overcomes the need to retrain the weights programmed onto the chip to recover accuracy losses. Hence, the weights to be programmed onto the chip only depend on the task and not on the particular chip being programmed, obviating the need for retraining the weights specific to each chip. We also demonstrated the integration of AIMC with some of the essential digital compute blocks and an on-chip communication fabric.

With additional digital circuitry that enables the layer-to-layer activation transfers and intermediate activation storage in local SRAM with appropriate appending and reorganization, it should be possible to run fully pipelined end-to-end inference workloads on the chip⁴³. This will be the subject of future work. This mixed-signal, heterogeneous compute architecture could also be extended to accommodate additional digital compute blocks (for example, attention compute blocks) to support additional neural networks such as transformers⁴². Further improvements in weight density will also be highly desirable for AIMC accelerators to be a strong competitor to existing digital solutions; this could be achieved by integrating the PCM devices closer to the transistor level at a denser pitch or via three-dimensional stacking of memory layers^{44,45}.

Methods

Chip and PCM device fabrication

Our experimental results are demonstrated on chips from 300 mm wafers fabricated with PCM inserted in a 14 nm backend of line at IBM Research at Albany NanoTech. The backend-of-line fabrication including PCM mushroom cell formation is done on top of the foundry-sourced frontend wafers. The PCM mushroom cell comprises a ring heater for the bottom electrode and a doped Ge₂Sb₂Te₃ and top electrode film stack, which is subtractively patterned to form the mushroom top. Wafers are characterized through several different in-line electrical tests to assess their quality before identifying wafers and die to send for dicing and packaging. Testing is done on discrete single structures embedded in the backend of line and also done on array diagnostic monitors with functional peripheral circuitry. The PCM device yield, even though very good (Fig. 2a), can be further improved by fabricating the chip in a single semiconductor fab or by enabling more thorough wafer-level testing before packaging.

Experimental platform

The experimental platform is built around a single chip packaged in a 1,525 pin ball grid array. Through a controlled collapse chip connection (C4), the chip can be mounted onto an interposer printed circuit board, which is placed on a socket for testing on a dedicated test board. For every core, 72 dedicated C4 pads provide analogue and digital supply

voltages in addition to a small number of pads used to implement a serial communication interface to control the core. The supply voltages of the cores in each row are grouped on the interposer. The resulting partition into eight power domains reduces the required number of components on the test board and maintain a high degree of redundancy and resilience against faulty cores, for example, due to manufacturing failures.

The chip is interfaced to a hardware platform comprising one field-programmable gate array module and a base board. The base board provides the chip socket, chip cooling, multiple power supplies per power domain as well as the voltage and current reference sources for the chip. A Trenz TE0808-04 module with Xilinx Zynq UltraScale+ field-programmable gate array is used to implement overall system control and data management as well as interface with the chip. The experimental platform is operated through Ethernet from a host computer, and a Python 3.8 environment is used to coordinate the experiments. The base board provides current measurement for all the chip supply rails by using high-side current sensing before the voltage regulators, allowing averaged current measurements with voltmeters.

Weight programming

The gate signals of the unit cells are connected in a diagonal fashion. The goal of this is to allow as many devices to be programmed in parallel as possible without causing an excessive current flow on any one BL or SL. During programming, only one gate signal is enabled (Extended Data Fig. 5a). These gate selection signals are controlled by the diagonal selection module.

The programming circuit controls the application of programming current profiles to the SLs of the crossbar array. There are a total of 32 parallel current digital-to-analogue converters (IDACs) that can apply digitally controlled current values to the SLs. Each IDAC is assigned to eight consecutive SLs of either negative or positive polarity (16 in total). A programming finite-state machine is in charge of controlling these IDACs from the configuration values written to its registers, as well as selecting the active SLs of the PCM devices to program.

To program a set of devices, a single diagonal must be selected before the IDACs are activated. This way, current can be applied to all the SLs in parallel without causing any current congestion in the chip. In practice, only 32 SLs are programmed in parallel due to area and power constraints. Therefore, it takes eight programming cycles to completely program one diagonal of devices for a particular polarity and device ID (1 or 2). Both square current pulses (RESET) and current pulses with a trailing edge (SET) are supported with configurable pulse widths and amplitudes. In the experiments presented here, we use a RESET pulse width of 125 ns with an amplitude of 700 μA, and a SET pulse width of 250 ns with a trailing edge of 50 ns and amplitude of 125 μA.

When programming a weight onto a unit cell, all the four PCM devices are first RESET and then the devices corresponding to the weight polarity are SET. For ODP, the SET pulses are sent only to device 1 and for TDP to devices 1 and 2. Thereafter, the device that receives the subsequent iterative programming pulses is selected. For ODP, device 1 is always selected. For TDP, if the target conductance value $|G_{\text{min}}|$ exceeds G_{SET} of both devices 1 and 2, the device with the lowest G_{SET} is selected for programming and the other is left as is. Otherwise, the device with the highest G_{SET} is selected for programming and the other is RESET. Extended Data Fig. 5b shows a diagram of the TDP algorithm.

Iterative programming involving a sequence of program-and-verify steps is then used to program the selected devices to the desired conductance values³⁰. After each programming pulse, a verify step is performed, and the value of the unit-cell conductance programmed in the preceding iteration is read. The read operation is performed by applying long PWM pulses of 0.2 V amplitude with 512 ns width and 256 ns precharge time to the selected diagonal, to ensure reliable read of the small individual conductance values by the ADCs. The programming current applied to the PCM device in the subsequent iteration is adapted according to the value of the error between the target level and read value of the unit-cell conductance. The unit-cell conductance is obtained by individually reading the conductance value of each PCM device, and computing $(g_1^+ + g_2^+) - (g_1^- + g_2^-)$. The programming pulses are square pulses of 125 ns width and amplitude varying between 125 and 700 μA . The programming sequence ends when the error between the target conductance and programmed conductance of the unit cell is smaller than a margin of 5 ADC counts or when the maximum number of iterations (30) has been reached.

ResNet-9 on CIFAR-10 training

The CIFAR-10 dataset³⁵ is a well-known image classification benchmark that comprises a total of 60,000 RGB images belonging to one of 10 classes. The dataset is split into a training (50,000 images) and test (10,000 images) set. Each image has a size of 32×32 .

We use ResNet-9 (ref. 46) since it is one of the most commonly used architectures for image classification. The architecture comprises eight convolutional layers (all followed by a batch norm and ReLU activation layer), which can be attributed to two main blocks. Each block has two convolutional layers and a residual layer that contains two other convolutional layers. The number of filters for each convolutional layer in order of appearance is [56, 112, 112, 112, 224, 224, 224, 224]. This network yields a total of 1,869,122 trainable parameters.

We first train a model without any hardware-aware training for $N = 300$ epochs and later, as part of the hardware-aware training, fine-tune the pretrained model for additional $N = 200$ epochs. Note that both training phases are fully executed off the chip in software. The batch size is set to $B = 128$. In both phases of training, we use stochastic gradient descent where the initial learning rate $\eta = 0.05$ is gradually reduced using a cosine annealing scheduler with $T_{\text{max}} = 200$. To improve performance, we randomly crop the training images to a shape of 32×32 (padding = 4) and apply a cutout⁴⁷ with a patch size of 4×4 . We apply the following methods during hardware-aware fine-tuning of the model. After every update, we clip the weights of each layer to a constant value ($\alpha = 1$) around zero. The tiling of the layers is simulated during training with a tile size of 256×256 . To make the model more robust to noise, we perturb the weights of each tile according to a model simulating two times the PCM programming noise derived experimentally in another work⁴⁸. Additionally, we add additive Gaussian noise on the output of each per-tile MVM with a standard deviation of 10% of the maximum output range. Finally, DACs and ADCs at the output of each tile are simulated during training with a resolution of 8 bits.

PTB-char LSTM network training

The PTB³⁶ corpus is a collection of articles that are commonly used for evaluating sequence labelling approaches, as well as the language

modelling capabilities of models on a per-word or per-character (PTB-char) level. We use PTB-char that has a vocabulary size of 50, that is, there are 50 different characters in the dataset. During preprocessing, every character is embedded into a random orthogonal vector (dimension $E = 128$) drawn from a standard Gaussian distribution. The training sequence of length 5,017,482 is split up into smaller sequences of length 150, which are shuffled after each epoch. The validation sequence is split up into smaller sequences of size 128, which we then use to determine the best model during training. The best model is finally evaluated on the whole test sequence of length 442,423. Since we are considering a language modelling task, the target sequence corresponds to the input sequence shifted to the left by one character. The model performance is evaluated using the BPC metric, which is the average cross-entropy loss evaluated with base-2 logarithm.

We use a one-layer LSTM⁴⁹, where the input gate projects the embedded inputs for each timestep to the LSTM cell (dimension $G = 2,016$). The projected input is then added to the equally shaped projection of the hidden state ($H = 504$). The LSTM cell then processes the combined input and produces the next cell and hidden state. The latter is used as an input to the output layer, which applies dropout⁵⁰ ($p = 0.25$) and maps the input to the output dimension, which is equal to the number of characters in the vocabulary. For the embedding, we use vectors sampled from a standard Gaussian that are orthogonalized using Gram–Schmidt orthogonalization. This results in dense, Gaussian-distributed embedded vectors that provide a strong signal with a high signal-to-noise ratio at the input of the ADC, instead of a more straightforward one-hot embedding that would result in poor signal-to-noise ratio when reading only a single row of unit cells with the ADC.

The training procedure is split into the standard training phase, followed by the hardware-aware fine-tuning. In the standard training phase, we train the model for $N = 200$ epochs using the Adam⁵¹ optimizer with an initial learning rate of $\eta = 0.01$, which is reduced by a multiplicative factor of $\gamma = 0.1$ every 50 epochs. We use a batch size of $B = 128$ and clip the gradients to have a maximum l_2 -norm of 5.0 to avoid training instabilities. After the standard training phase, we set the dropout probability to $p = 0$ and inject Gaussian noise into the weights during each forward pass¹². Given the weight matrix W , each weight W_{ij} follows the distribution $\mathcal{N}(W_{ij}, \zeta \times |W_{\text{max}} - W_{\text{min}}|)$, where $\zeta = 0.075$ controls the amount of noise relative to the dynamic range of the weights. To avoid weight outliers, we clip the weights of each layer to $\alpha = 2.0$ times the layers' standard deviation. We find that for randomly distributed timesteps, the inputs to the hidden gate contain performance-critical outliers that cannot be clipped post-training. We, therefore, also clip the inputs to the hidden gate (which are also the outputs of the LSTM cell) to a constant value of $\beta = 0.06$.

LSTM network for image caption generation training

We use Flickr 8k (ref. 37), which is a collection of images from Flickr with five human-generated captions per image. The dataset is split into a training, validation and test set with 6k, 1k and 1k samples, respectively. Due to size limitations incurred by the hardware, we reduce the vocabulary size from the original 8,918 to 4,064 (16 cores with an output size of 254) by removing the least frequently used words. As a consequence, 12% of the available captions are eliminated. Nine images are removed from the dataset altogether because every caption contains an eliminated word. Using BLEU-4, which is the most common BLEU variant, we verified that removing 12% of the captions did not substantially alter the performance of the network ($0.1609 \rightarrow 0.1557$ for the whole and reduced dataset, respectively). As a final preprocessing step, we apply basic tokenization on the sequences.

Similar to another work⁵², we extract 2,048 features from each image using an InceptionV3 (ref. 53) model that was pretrained on ImageNet⁵⁴. The extracted features are then projected onto a 504-dimensional subspace, followed by batch normalization⁵⁵. As a

generative model, we use a one-layer LSTM⁴⁹ architecture (dimension $G = 2,016$). At time $t = 0$, the LSTM receives as input the embedded image features and an all-zero hidden-state vector. At time $t > 0$, the LSTM receives as input the embedded word (learnable embedding vectors initialized with uniform distribution) and hidden state that was generated in the previous timestep. At each timestep, the output is generated by passing the 504-dimensional hidden state of the LSTM through a dense layer, producing an output vector of shape 4,064, over which a softmax is computed. At each timestep, the word with the highest softmax probability is chosen. The LSTM runs for at most 24 timesteps or until an end-of-sequence token is generated.

We first train a network without any hardware-aware training for $N = 70$ epochs using a batch size of $B = 120$. We use the Adam⁵¹ optimizer with an initial learning rate of $\eta = 0.001$. To avoid overfitting, we apply dropout to the activations of the input layer with a dropout probability of $p = 0.5$. We then fine-tune the pretrained model for another 50 epochs using hardware-aware training. During this phase, we inject Gaussian noise (the same type as for PTB-char; $\zeta = 0.080$) and evaluate with slightly less noise ($\zeta = 0.035$). After every update, we clip the weights to $\alpha = 2.0$ standard deviations and apply l_2 -regularization with $\lambda = 1 \times 10^{-4}$. To avoid exploding gradients, we clip the gradient l_2 -norm to 0.5. During training, we maximize the sum of the negative log likelihood of the correct word at each step. As an evaluation metric, we use the BLEU⁴¹ score, which measures the precision of word n -grams between a generated and multiple reference captions. We report the n -gram BLEU score for $n \in \{1, 2, 3, 4\}$.

Weight mapping onto cores and functional modelling

The mapping of weights onto the chip is done by converting a weight matrix for each layer into 256×256 conductance submatrices, one for each core the layer is mapped onto. The weight matrix is split into the smallest possible amount of submatrices that have the same number of rows and columns, both less than 256. For example, the $2,016 \times 224$ weight matrix of a deep layer of ResNet-9 is converted into eight 252×224 submatrices. Each submatrix is then reshaped into a 256×256 matrix by zero filling. Finally, the weight values of each submatrix are converted into conductance values using equation (1), with $G_{\max}$ being determined on a per-core basis. The $G_{\max}$ value of a core is set to the minimum value among the maximum conductance supported by the unit cell (80 ADC counts for ODP and 160 ADC counts for TDP; Fig. 2a) and the maximum conductance that leads to currents of all the BLs to be less than the maximum current supported by the ADC (approximately 100 μA). For higher BL currents, the ADC response starts to saturate, which introduces nonlinearity in the resulting MVMs and therefore must be avoided. This additional constraint means that for some layers, the full unit-cell conductance range cannot be utilized. For ODP, the $G_{\max}$ value used for all the networks is 80 except for the image captioning network, where the hidden gate layer uses 70 and the output-layer values are between 40 and 80 depending on the core. For ResNet-9 with TDP, we use $G_{\max}$ of 160 for Conv0 and dense layers, values between 80 and 120 for Conv1–4 and values between 120 and 140 for Conv5–7. For PTB-char with TDP, we use $G_{\max}$ of 100 for the LSTM unit and 120 for the output layer. Therefore, only ResNet-9 takes notable advantage of TDP, because the lower utilization of the cores (especially for Conv0 and dense layers) allows the use of larger $G_{\max}$ values that improve the MVM accuracy.

Given the conductance mapping provided by the above procedure, we built a functional model of the chip that includes the quantization from the digital peripheral circuits and the hardware-measured weight noise. The quantization model includes the quantization resulting from all the data conversions performed by the chip during inference execution. It includes the 8-bit input quantization from PWM, 12-bit quantization from ADC, 8-bit data conversion from LDPU and a GDPU functional model (including the discrete tanh LUTs and 8-bit output data conversion). The weight noise model includes the per-core weight

error measured when performing MVMs with the network weights being programmed. We perform multiple MVMs on each core and compute the resulting weight error $\text{std}(W - \hat{W})/W_{\max}$, where $\text{std}(W)$ is the standard deviation computed over all the elements of W (Fig. 2c). We then fit the experimental weight error data points as a function of the target weight with a polynomial function for each core. Weight noise during inference is simulated by injecting Gaussian noise into the synaptic weights with a weight-dependent standard deviation given by this polynomial function.

Power measurements

We measured the chip energy efficiency and latency for four different use cases. The first one is the chip performing only MVMs at full utilization (all 64 cores active in parallel), which provides the maximum MVM efficiency of the chip that can be compared with other works. The reported MVM energy includes the PCM crossbar array, ADCs and PWMs (operations performed by the LDPU and GDPU used in neural network executions, such as activation functions, are not included in the MVM energy). The second use case is the processing of one input to a deep layer of ResNet-9 that spans eight cores, including the MVMs, data aggregation with on-chip links and LDPU operations. The third and fourth use cases are the processing of one timestep of the LSTM unit of the PTB and image caption generation networks, respectively, including the MVMs, data aggregation with on-chip links, LDPU and GDPU operations.

For each use case, the power consumed by the chip is measured both in the standby mode (static) and performing the computations (dynamic) to calculate the energy. The average current drawn by the ADC and digital supplies operating at 0.85 V is measured using voltmeters (Fluke models 87V and 185). The dynamic power consumption is obtained for all the monitored supplies by sending a stream of MVM commands with a period of 5 μs , and measuring the averaged power during continuous operation of a few seconds. The dynamic energy is computed by multiplying the dynamic power by the MVM command period. The total energy is calculated by adding the static and dynamic contributions for all the monitored supplies. The latency is computed by running the register transfer level code for each use case.

Data availability

The data that support the plots within this paper and other findings of this study are available from the corresponding authors upon reasonable request.

References

- Murmann, B. Mixed-signal computing for deep neural network inference. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **29**, 3–13 (2021).
- Shafiee, A. et al. ISAAC: a convolutional neural network accelerator with in-situ analog arithmetic in crossbars. in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)* 14–26 (IEEE Press, 2016).
- Sebastian, A., Le Gallo, M., Khaddam-Aljameh, R. & Eleftheriou, E. Memory devices and applications for in-memory computing. *Nat. Nanotechnol.* **15**, 529–544 (2020).
- Yu, S., Jiang, H., Huang, S., Peng, X. & Lu, A. Compute-in-memory chips for deep learning: recent trends and prospects. *IEEE Circuits Syst. Magazine* **21**, 31–56 (2021).
- Lanza, M. et al. Memristive technologies for data storage, computation, encryption, and radio-frequency communication. *Science* **376**, eabj9979 (2022).
- Wang, Z. et al. Resistive switching materials for information processing. *Nat. Rev. Mater.* **5**, 173–195 (2020).
- Xiao, T. P., Bennett, C. H., Feinberg, B., Agarwal, S. & Marinella, M. J. Analog architectures for neural network acceleration based on non-volatile memory. *Appl. Phys. Rev.* **7**, 031301 (2020).

8. Yu, S. et al. Binary neural network with 16 Mb RRAM macro chip for classification and online training. in *2016 IEEE International Electron Devices Meeting (IEDM)* 16.2.1–16.2.4 (IEEE, 2016).
9. Hu, M. et al. Memristor-based analog computation and neural network classification with a dot product engine. *Adv. Mater.* **30**, 1705914 (2018).
10. Tsai, H. et al. Inference of long-short term memory networks at software-equivalent accuracy using 2.5m analog phase change memory devices. in *2019 Symposium on VLSI Technology T82–T83* (IEEE, 2019).
11. Yao, P. et al. Fully hardware-implemented memristor convolutional neural network. *Nature* **577**, 641–646 (2020).
12. Joshi, V. et al. Accurate deep neural network inference using computational phase-change memory. *Nat. Commun.* **11**, 2473 (2020).
13. Biswas, A. & Chandrakasan, A. P. CONV-SRAM: an energy-efficient SRAM with in-memory dot-product computation for low-power convolutional neural networks. *IEEE J. Solid-State Circuits* **54**, 217–230 (2019).
14. Merrikh-Bayat, F. et al. High-performance mixed-signal neurocomputing with nanoscale floating-gate memory cell arrays. *IEEE Trans. Neural Netw. Learn. Syst.* **29**, 4782–4790 (2018).
15. Cai, F. et al. A fully integrated reprogrammable memristor–CMOS system for efficient multiply–accumulate operations. *Nat. Electron.* **2**, 290–299 (2019).
16. Chen, W.-H. et al. CMOS-integrated memristive non-volatile computing-in-memory for AI edge processors. *Nat. Electron.* **2**, 420–428 (2019).
17. Yin, S., Sun, X., Yu, S. & Seo, J.-S. High-throughput in-memory computing for binary deep neural networks with monolithically integrated RRAM and 90-nm CMOS. *IEEE Trans. Electron Devices* **67**, 4185–4192 (2020).
18. Khaddam-Aljameh, R. et al. HERMES-Core—a 1.59-TOPS/mm² PCM on 14-nm CMOS in-memory compute core using 300-ps/LSB linearized CCO-based ADCs. *IEEE J. Solid-State Circuits* **57**, 1027–1038 (2022).
19. Deaville, P., Zhang, B. & Verma, N. A 22nm 128-kb MRAM row/column-parallel in-memory computing macro with memory-resistance boosting and multi-column ADC readout. in *2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)* 268–269 (IEEE, 2022).
20. Khwa, W.-S. et al. A 40-nm, 2M-cell, 8b-precision, hybrid SLC-MLC PCM computing-in-memory macro with 20.5–65.0TOPS/W for tiny-AI edge devices. in *2022 IEEE International Solid-State Circuits Conference (ISSCC)* **65**, 1–3 (IEEE, 2022).
21. Wan, W. et al. A compute-in-memory chip based on resistive random-access memory. *Nature* **608**, 504–512 (2022).
22. Hung, J.-M. et al. A four-megabit compute-in-memory macro with eight-bit precision based on CMOS and resistive random-access memory for AI edge devices. *Nat. Electron.* **4**, 921–930 (2021).
23. Fick, L., Skrzyniarz, S., Parikh, M., Henry, M. B. & Fick, D. Analog matrix processor for edge AI real-time video analytics. in *2022 IEEE International Solid-State Circuits Conference (ISSCC)* **65**, 260–262 (IEEE, 2022).
24. Jia, H. et al. Scalable and programmable neural network inference accelerator based on in-memory computing. *IEEE J. Solid-State Circuits* **57**, 198–211 (2022).
25. Narayanan, P. et al. Fully on-chip MAC at 14 nm enabled by accurate row-wise programming of PCM-based weights and parallel vector-transport in duration-format. *IEEE Trans. Electron Devices* **68**, 6629–6636 (2021).
26. Dazzi, M. et al. Efficient pipelined execution of CNNs based on in-memory computing and graph homomorphism verification. *IEEE Trans. Comput.* **70**, 922–935 (2021).
27. Boybat, I. et al. Neuromorphic computing with multi-memristive synapses. *Nat. Commun.* **9**, 2514 (2018).
28. Khaddam-Aljameh, R. et al. A multi-memristive unit-cell array with diagonal interconnects for in-memory computing. *IEEE Trans. Circuits Syst. II, Exp. Briefs* **68**, 3522–3526 (2021).
29. Sarwat, S. G. et al. Mechanism and impact of bipolar current voltage asymmetry in computational phase-change memory. *Adv. Mater.* 2201238 (2022).
30. Papandreou, N. et al. Programming algorithms for multilevel phase-change memory. in *IEEE International Symposium on Circuits and Systems (ISCAS)* 329–332 (IEEE, 2011).
31. Le Gallo, M. et al. Precision of bit slicing with in-memory computing based on analog phase-change memory crossbars. *Neuromorp. Comput. Eng.* **2**, 014009 (2022).
32. Ielmini, D., Sharma, D., Lavizzari, S. & Lacaíta, A. Reliability impact of chalcogenide-structure relaxation in phase-change memory (PCM) cells, part I: experimental study. *IEEE Trans. Electron Devices* **56**, 1070–1077 (2009).
33. Le Gallo, M., Krebs, D., Zipoli, F., Salinga, M. & Sebastian, A. Collective structural relaxation in phase-change memory devices. *Adv. Electron. Mater.* **4**, 1700627 (2018).
34. Le Gallo, M., Sebastian, A., Cherubini, G., Giefers, H. & Eleftheriou, E. Compressed sensing with approximate message passing using in-memory computing. *IEEE Trans. Electron Devices* **65**, 4304–4312 (2018).
35. Krizhevsky, A. *Learning Multiple Layers of Features from Tiny Images*. Tech. Rep. (2009).
36. Marcus, M. P., Santorini, B. & Marcinkiewicz, M. A. Building a large annotated corpus of English: the Penn Treebank. *Comput. Linguist.* **19**, 313–330 (1993).
37. Rashtchian, C., Young, P., Hodosh, M. & Hockenmaier, J. Collecting image annotations using Amazon’s mechanical turk. in *Proc. NAACL HLT 2010 Workshop on Creating Speech and Language Data with Amazon’s Mechanical Turk CSLDAMT ’10* 139–147 (Association for Computational Linguistics, 2010).
38. Rasch, M. J. et al. Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators. Preprint at *arXiv* <https://doi.org/10.48550/arXiv.2302.08469> (2023).
39. Rasch, M. J. et al. A flexible and fast pytorch toolkit for simulating training and inference on analog crossbar arrays. in *2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems (AICAS)* 1–4 (IEEE, 2021).
40. Mujika, A., Meier, F. & Steger, A. Fast-slow recurrent neural networks. in *Advances in Neural Information Processing Systems* Vol. 30 (eds Guyon, I. et al.) (Curran Associates, 2017).
41. Papineni, K., Roukos, S., Ward, T. & Zhu, W.-J. Bleu: a method for automatic evaluation of machine translation. in *Proc. 40th Annual Meeting on Association for Computational Linguistics, ACL ’02* 311–318 (Association for Computational Linguistics, 2002).
42. Jain, S. et al. A heterogeneous and programmable compute-in-memory accelerator architecture for analog-AI using dense 2-D mesh. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **31**, 114–127 (2022).
43. Dazzi, M., Sebastian, A., Benini, L. & Eleftheriou, E. Accelerating inference of convolutional neural networks using in-memory computing. *Front. Comput. Neurosci.* **15**, 674154 (2021).
44. Lin, P. et al. Three-dimensional memristor circuits as complex neural networks. *Nat. Electron.* **3**, 225–232 (2020).
45. Huo, Q. et al. A computing-in-memory macro based on three-dimensional resistive random-access memory. *Nat. Electron.* **5**, 469–477 (2022).

46. He, K., Zhang, X., Ren, S. & Sun, J. Deep residual learning for image recognition. in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* 770–778 (IEEE Computer Society, 2016).
47. Devries, T. & Taylor, G. W. Improved regularization of convolutional neural networks with dropout. Preprint at *arXiv* <https://doi.org/10.48550/arXiv.1708.04552> (2017).
48. Nandakumar, S. R. et al. Phase-change memory models for deep learning training and inference. in *2019 26th IEEE International Conference on Electronics, Circuits and Systems (ICECS)* 727–730 (IEEE, 2019).
49. Hochreiter, S. & Schmidhuber, J. Long short-term memory. *Neural Comput.* **9**, 1735–1780 (1997).
50. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.* **15**, 1929–1958 (2014).
51. Kingma, D. P. & Ba, J. Adam: a method for stochastic optimization. in *3rd International Conference on Learning Representations, ICLR 2015* (2015).
52. Vinyals, O., Toshev, A., Bengio, S. & Erhan, D. Show and tell: a neural image caption generator. in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* 3156–3164 (IEEE, 2015).
53. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J. & Wojna, Z. Rethinking the inception architecture for computer vision. in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* 2818–2826 (IEEE, 2016).
54. Russakovsky, O. et al. Imagenet large scale visual recognition challenge. *Int. J. Comput. Vision* **115**, 211–252 (2015).
55. Ioffe, S. & Szegedy, C. Batch normalization: accelerating deep network training by reducing internal covariate shift. in *Proc. 32nd International Conference on International Conference on Machine Learning—Volume 37, ICML'15* 448–456 (PMLR, 2015).

Acknowledgements

We thank G. W. Burr, M. Bühler, T. Maurer, A. Müller, Y. Kohda, K. Hosakawa, S. Ambrogio, F. L. Lie, F. Liu, T. Levin and T. Gordon for assistance with the chip design; A. Okazaki, H. Mori and M. Bergendahl for assistance with the chip packaging; J. F. Mas, G. Cristiano and J. Paret for chip testing and simulation; F. Odermatt, I. Boybat, S. R. Nandakumar, C. Piveteau, C. Lammie and H. Benmeziane for help with the network deployment on the chip; and A. Pantazi, R. Haas, A. Curioni, S. Tsai, W. Haensch, J. Burns, R. Divakaruni and M. Khare for managerial support. We would also like to thank L. Benini and B. Rajendran for their support with supervising the students. This work was supported by the IBM Research AI Hardware Center. A. Sebastian acknowledges partial funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (grant agreement nos. 682675 and 966764).

Author contributions

M.L.G. and A. Sebastian defined the neural network inference and compute precision characterization research. R.K.-A. led and performed the analogue design of the chip. M.S. led the digital design of the chip. M.S., M.D., G.K., M.B., A. Singh, S.M.M. and P.A.F. performed the digital design. M.L.G., A.V., B.K., G.K. and A.G. performed the chip testing and wrote the code to operate it. A.V. and B.K. performed the neural network inference and MVM characterization hardware experiments. J.B., X.T., V.J. and M.J.R. performed the hardware-aware training of the neural networks. M.L.G. and U.E. performed the chip performance measurements. U.E. built the chip testing platform. A.P. and T.A. wrote the field-programmable gate array code to interface with the chip. K.B., S.C., I.O., T.P., V.C., C.S., I.A. and N.S. performed the backend integration of the PCM devices and wafer-level testing. M.L.G. wrote the manuscript with input from all authors. V.N., P.A.F., E.E. and A. Sebastian supervised the project.

Competing interests

The authors declare no competing interests.

Additional information

Extended data is available for this paper at <https://doi.org/10.1038/s41928-023-01010-1>.

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41928-023-01010-1>.

Correspondence and requests for materials should be addressed to Manuel Le Gallo or Abu Sebastian.

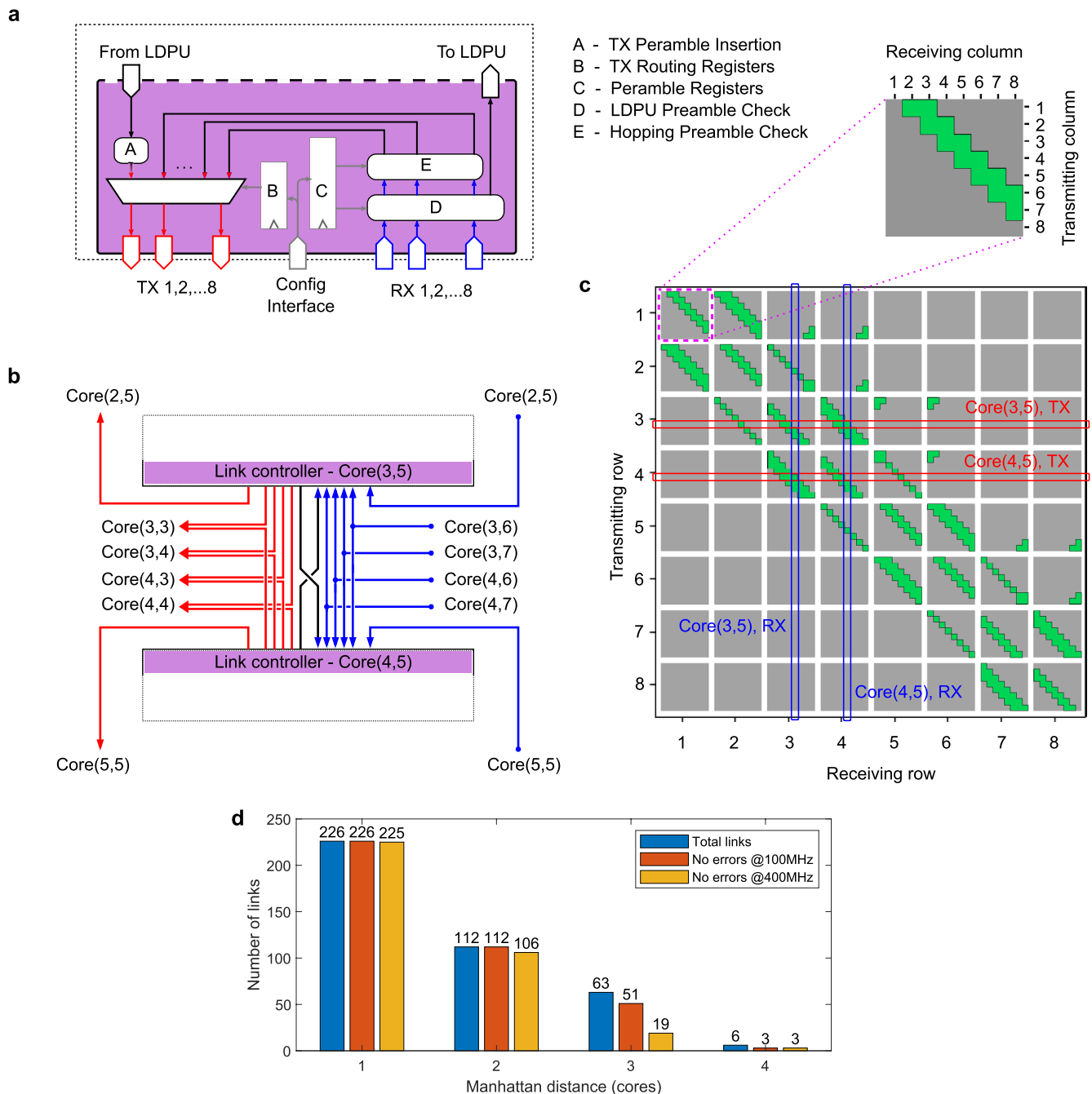
Peer review information *Nature Electronics* thanks the anonymous reviewers for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

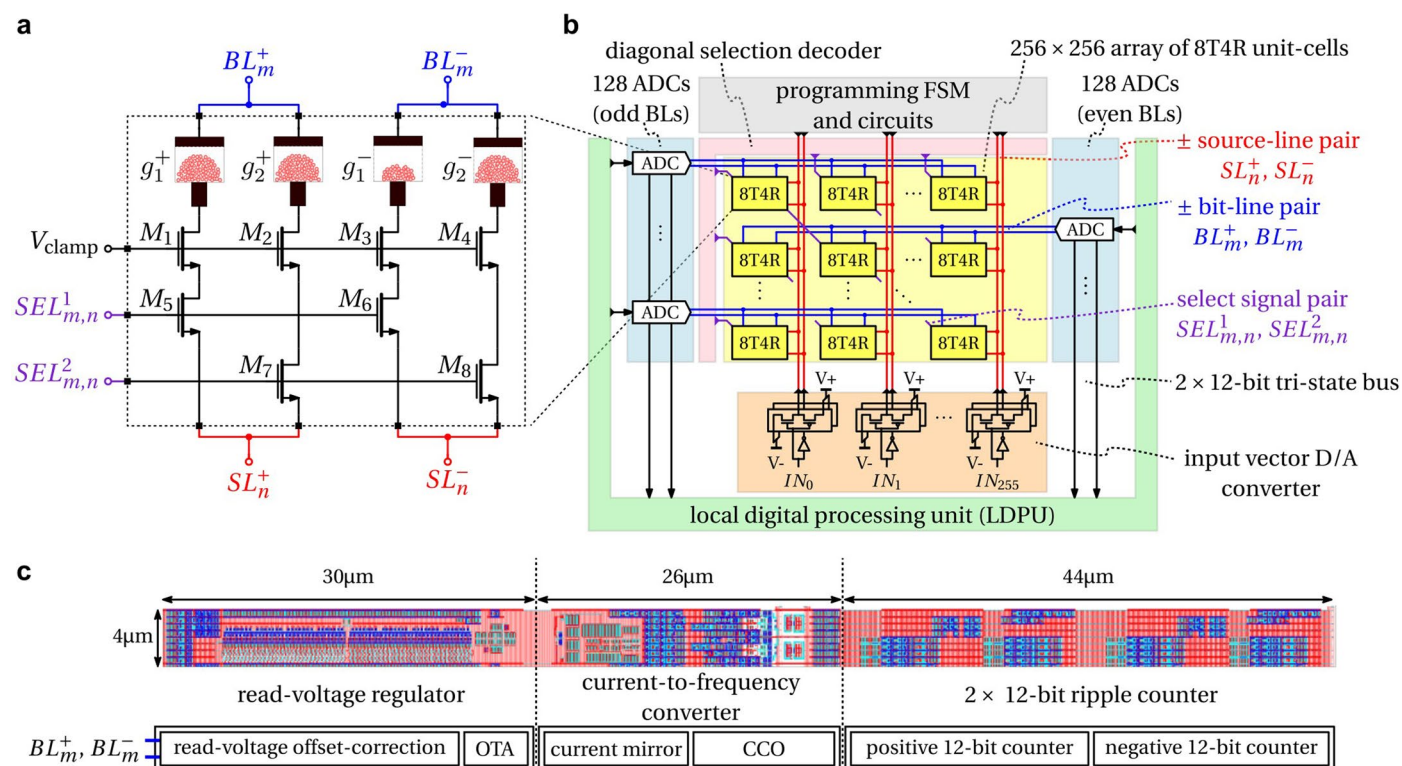
Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© The Author(s), under exclusive licence to Springer Nature Limited 2023



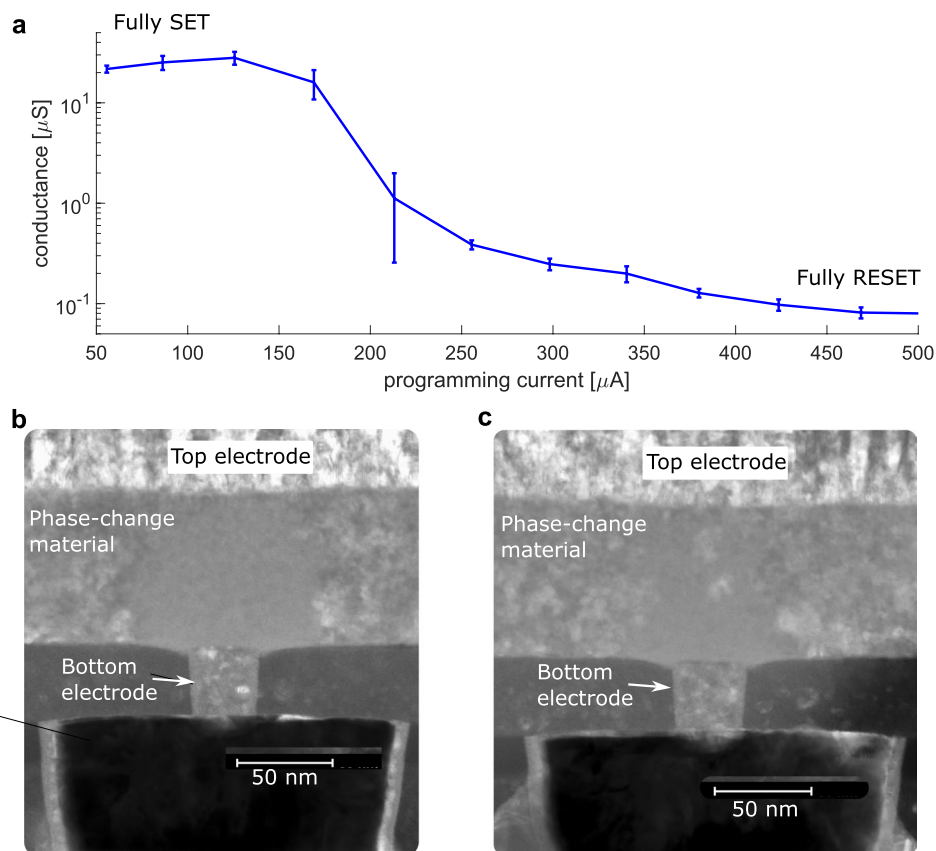
Extended Data Fig. 1 | Digital communication fabric. **a**, Schematic of link controller. The dotted line refers to the core boundary. The transmitter-side link controller prepends a preamble to the payload being sent on the links. The preamble contains information to uniquely identify a routing path for a particular type of payload. If the receiving core is enabled to receive data from the transmitting core in the routing table, the link controller samples the incoming data. Furthermore, the link controller in the transmitting and receiving cores can select a portion of the payload according to another set of routing registers. **b**, Possible link connections for Core(3,5) and Core(4,5), where the notation Core(r,c) refers to the core located at row r and column c in Fig. 1b. **c**, Link connections for the entire chip (available connections are denoted in green color). The RX and TX connections for Core(3,5) and Core(4,5) shown in **b** are

indicated. **d**, Link characterization results on one chip for communicating data from the LDPU of a core to the LDPU of another core. A payload of 255 bytes is sent by the transmitter core and an error is triggered if at least one byte of the payload received in the LDPU of the receiver core does not match with the original payload. All links with a Manhattan distance of 1 or 2 cores show no errors when run at 100 MHz, and 98% of them at 400 MHz. Links with longer Manhattan distances show more errors potentially due to attenuation from longer-distance routing metal wires due to parasitics. The issue can be mitigated in a future design by placement of buffers at reasonable distances along the wires, or by employing a core connectivity matrix that does not rely on long distance links. All the links used in the experimental demonstrations shown in this work have a Manhattan distance of 1 core and are fully working at 400 MHz.



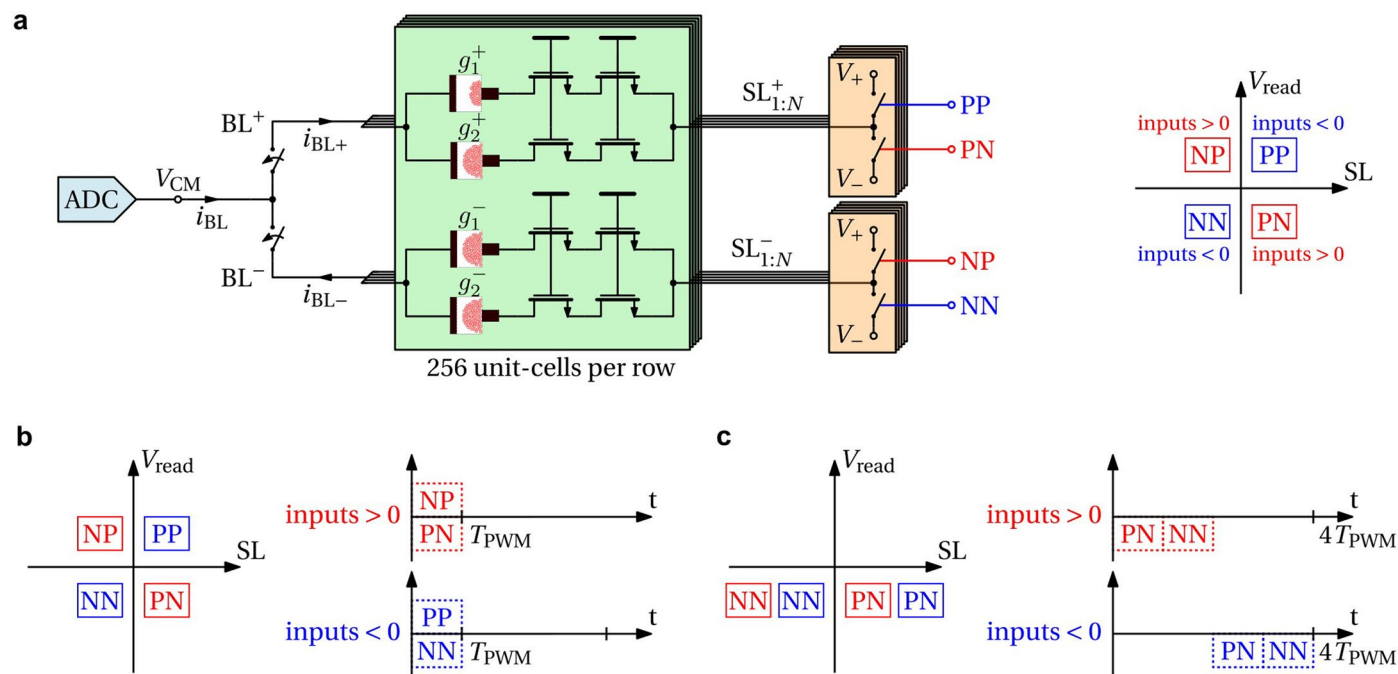
Extended Data Fig. 2 | PCM crossbar array. **a**, Schematic of 8T4R unit-cell. The top electrodes of the conductance pairs of each polarity connect to separate bit lines BL_m^+ , BL_m^- and the sources of their lower access-transistors connect to separate source lines SL_n^+ , SL_n^- . Thus, the devices in a conductance pair are weighted with equal significance and the total conductance per unit-cell becomes: $(g_1^+ + g_2^+) - (g_1^- + g_2^-)$. **b**, Schematic of PCM crossbar array. To program the PCM devices, the dedicated per-core programming FSM instructs the diagonal selection decoder to enable one diagonal of cells that contains the devices that are to be programmed. The diagonal selection decoder controls the $SEL_{m,n}^1$ and $SEL_{m,n}^2$ signals in the unit-cell, which are routed diagonally

throughout the array. The selected devices are programmed by the current-steering DAC-based programming units located on top of the PCM array. To perform an MVM, the 256 inputs to the crossbar array ($IN_0 - IN_{255}$) are applied via the red source lines (SLs) to the 8T4R cells. The resulting bit line (BL) currents are summed up on the blue wires and read by the ADCs that flank the crossbar array on the left and right. **c**, Layout of one ADC. The block diagram that is shown below the layout illustrates the various components of the ADC, namely, the read voltage regulator, the current-to-frequency converter, and the 2×12 -bit ripple counter.



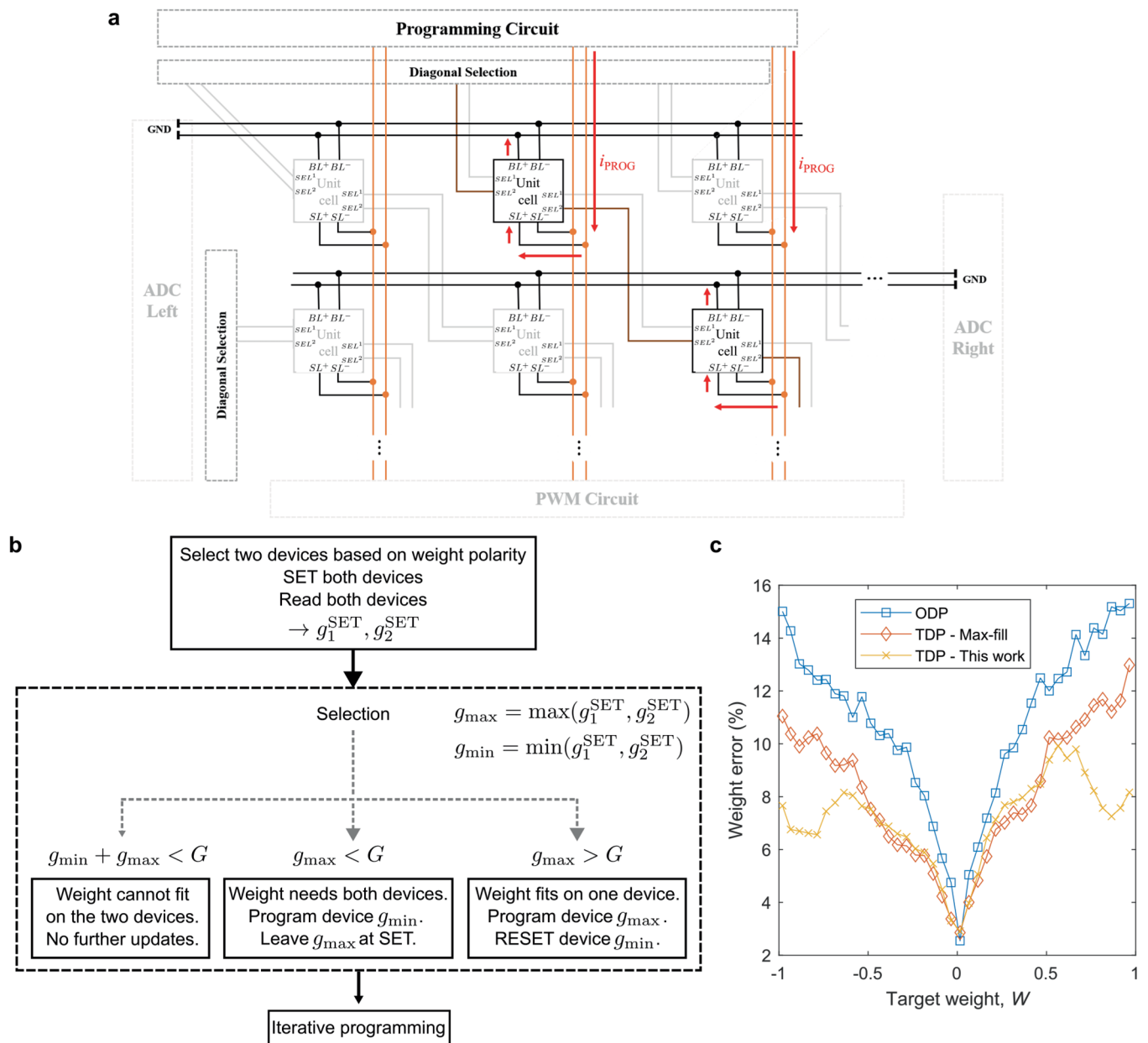
Extended Data Fig. 3 | PCM device. **a**, A typical programming curve indicating the programmed device conductance as a function of the programming current. The device conductance is determined by the phase configuration within the PCM device and in particular, the size of the amorphous region. Data are presented as mean values $\pm$ one standard deviation over 10 repeated measurements on a single device. **b**, Low-angle annular darkfield (LAADF)

scanning transmission electron microscope (STEM) image of a fully RESET PCM device showing a substantially large amorphous region that fully blocks the bottom electrode. LAADF enables the imaging of the amorphous region with high resolution. **c**, LAADF of a partially RESET PCM device showing a much smaller amorphous region. The synaptic weights are stored in an analog manner in terms of these phase configurations and the resulting conductance values.



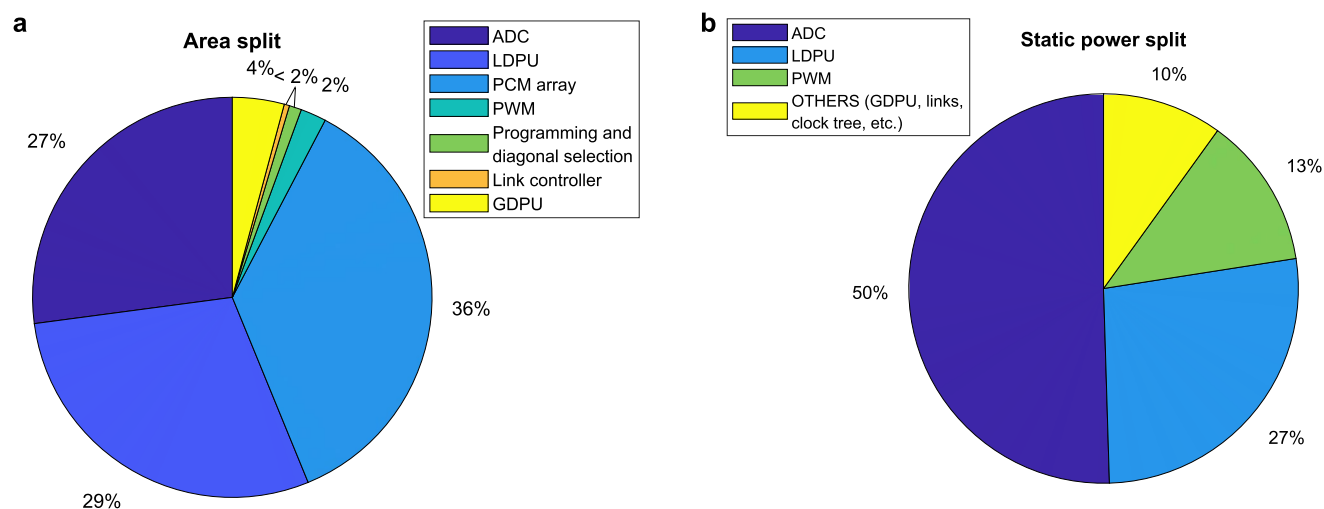
Extended Data Fig. 4 | Input modulation modes for MVM. a, Full array read procedure for MVMs showing the connection between ADC, unit-cells, and input modulator switches. Signals PP and PN connect the positive source lines $SL_{1:N}^+$ to the positive potential V_+ and negative potential V_- , respectively. For NP and NN it is vice versa. **b**, 1-phase modulation mode. Inputs of positive and negative

polarity are applied to weights of positive and negative polarity in one modulation cycle T_{PWM} . **c**, 4-phase modulation mode. Inputs of positive and negative polarity are applied individually to weights of positive and negative polarity in four modulation cycles.



Extended Data Fig. 5 | Weight programming procedure. a, Crossbar array during programming. **b**, Proposed TDP algorithm to program a target conductance value G on a unit-cell. **c**, Weight error comparison between TDP of this work and previous approaches. TDP - Max-fill refers to programming the two devices with iterative programming up to the ODP $G_{\max}$, as proposed in Ref. 31. Due to the wide SET distribution shown in Fig. 2a, some devices in the core either

cannot achieve $G_{\max}$, or conversely could be programmed to much higher conductance values than $G_{\max}$. Therefore, the latter approach leads to programming inaccuracies resulting from either under-utilizing the conductance range of individual devices or from devices that cannot reach $G_{\max}$. The proposed TDP algorithm solves this issue by using the readout SET conductance of the devices of the unit-cell to map the weight.



Extended Data Fig. 6 | Area and power splits. **a**, Area breakdown of the main chip components. **b**, Static power consumed by the different components of the chip measured for the operation of the LSTM unit of the image caption generation task (4 core rows and GDPU active).

Extended Data Table 1 | Summary of IBM HERMES Project Chip specifications

Name	Value
Technology	14-nm CMOS + IBM PCM
Supply voltage	0.85 – 0.95 V
Chip Area	$12\text{ mm} \times 12\text{ mm} = 144\text{ mm}^2$
Single core area	$1.2\text{ mm} \times 1.16\text{ mm} = 1.392\text{ mm}^2$
MVM area per core	$0.870\text{ mm} \times 0.730\text{ mm} = 0.635\text{ mm}^2$
GDPU macro area	$0.809\text{ mm} \times 0.347\text{ mm} = 0.281\text{ mm}^2$
Number of cores	64
Number of GDPUs	8
Number of C4 pads per core	72
Total number of C4 pads	5,616
External contact pads	1,520
Maximum MVM clock frequency	1 GHz
Maximum LDPU/GDPU clock frequency	400 MHz
Maximum link data-rate	3.2 Gbit/s
Number of unit-cells per core	$256 \times 256 = 65,536$
Number of PCM devices per unit-cell	4
Number of PCM devices per core	$256 \times 256 \times 4 = 262,144$
Number of PCM devices per chip	$256 \times 256 \times 4 \times 64 = 16,777,216$
Size of PCM-based unit-cell	$1.854\text{ }\mu\text{m} \times 2.208\text{ }\mu\text{m}$
Nominal PCM resistance range per PCM device	100 k Ω – 10 M Ω
Resolution of the time-encoded inputs	8 bit (1 ns/LSB)
Read voltage range	100 – 400 mV
Number of ADCs per core	256 (1 per row)
ADC output bits	$2 \times 12\text{ bit}$
Number of write heads per core	32
Max. supported write current per write-head	800 μA

Extended Data Table 2 | Comparison of IBM HERMES Project Chip with other multi-core AIMC chips demonstrating neural network inference

	This work		ISSCC '22 [20]	Nature '22 [21]	Nat. Elec. '21 [22]	ISSCC '22 [23]	JSSC '22 [24]
	1-phase read	4-phase read					
CMOS technology	14 nm		40nm	130 nm	22 nm	40 nm	16nm
AIMC device	PCM		PCM	RRAM	RRAM	nor-Flash	SRAM
Chip area (mm ²)	144		18	159	6	190	25
Number of cores (core size)	64 (256x256)		8 (256x1024)	48 (256x256)	8 (1024x512)	76 (1024x1024)	16 (1152x256)
Number of weights	4.2M		400k	1.6M	524k	80M	1.2M
Input/weight/output precision	8b/Analog/8b		8b/8b/19b	4b/Analog/6b	8b/8b/14b	8b/Analog/8b	4b/4b/8b
CIFAR-10 accuracy	-	92.81%	91.89%	85.66%	92.01%	-	91.51%
Peak MVM throughput (TOPS)	63.1	16.1	0.475	0.754	0.0337	16.6	11.8
MVM TOPS/W	9.76	2.48	20.5	16	15.6	5.2	121
MVM GOPS/mm ²	1550	400	26.4	4.7	5.61	-	2670
Energy-delay product of a 256 × 256 MVM (J·s·10 ⁻¹⁵)	1.8	27.5	16.3	32.0	65.4	60.5	0.22
Non-MVM operations supported	ReLU/sigmoid/tanh, batch norm., LSTM hidden state, partial sum accumulation		-	ReLU/sigmoid/tanh	-	All (1 RISC-V processor per core)	All CNN/LSTM operations (SIMD ALU + LUTs)